Yet Another Smacof - Square Symmetric Case

Jan de Leeuw

October 25, 2026

We rewrite the metric/nonmetric and weighted/unweighted versions of the smacof program for square symmetric data as one monolithic C program. R is used for taking care of the data and parameter setup, the I/O, and of issuing a single call to *.C()* to load the shared library that does all the computations. This makes this new *smacofSS()* program five to fifty times as fast (for our examples) as the *smacofSym()* function from the R smacof package. In addition we included various options to accelerate convergence. Utilities for various initial configurations and plots are included in the package. Examples are included to compare output and time for the R and C versions and to illustrate the plots and the effect of the various acceleration strategies.

Table of contents

1 Introduction **3**

2 MDS Data Structures **4**

3 Computation **8**

4 Software **9**
4.1 Arguments 9
4.2 Value 10

5 Details **11**
5.1 Interface C and R 11

5.2 Majorization . . . 11
5.3 Monotone Regression . . . 12
5.4 Initial Configuration . . . 12
5.4.1 Torgerson . . . 13
5.4.2 Guttman . . . 13
5.4.3 Elegant . . . 14
5.4.4 Full-dimensional Scaling . . . 14
5.5 Acceleration . . . 14

6 Utilities **15**
6.1 Data Utilities . . . 15
6.2 Plotting . . . 16
6.3 Newton . . . 16

7 Example Data Sets **16**
7.1 Ekman Data . . . 17
7.2 Morse Data . . . 17
7.3 Gruijter Data . . . 17
7.4 Wish Data . . . 17
7.5 Iris Data . . . 17

8 Comparisons **17**
8.1 Initial . . . 17
8.2 Outcome . . . 18
8.3 Time . . . 19

9 Comparing Strategies **21**
9.1 Data . . . 21
9.2 Number of Iterations . . . 21
9.2.1 Strategy (x) . . . 21
9.2.2 Strategy (0,x) . . . 22
9.3 Timing . . . 22
9.3.1 Strategy (x) . . . 22
9.3.2 Strategy (0,x) . . . 23
9.4 Nonmetric . . . 24

10 Plots **25**

References **41**

Note: This is a working manuscript which may be expanded/updated frequently. All suggestions for improvement are welcome. All Rmd, tex, html, pdf, R, and C files are in the

public domain. Attribution will be appreciated, but is not required. All files can be found at https://github.com/deleeuw/smacofFlat

1 Introduction

In Multidimensional Scaling (MDS) we start with n *objects*. Objects can be anything: people, animals, molecules, locations, time points, stimuli, political parties, and so on. We have information about the *similarities* or *dissimilarities* of some or all of the $\binom{n}{2}$ pairs of objects. Say we have information about m dissimilarities δ_k, with $k = 1, \cdots m$. Thus each index k refers to a pair of indices (i, j), with $1 \leq i \leq n$ and $1 \leq j \leq n$. The information is of the form $\delta \in \Delta$, where Δ is a known subset of the non-negative orthant of $\mathbb{R}^m$. If the dissimilarities are known numbers and Δ has only a single element the MDS problem is *metric*, in all other cases the problem is *non-metric*.

In MDS we want to map the objects into *points* in p-dimensional Euclidean space in such a way that the distances d_k between the points approximate the dissimilarities δ_k. The quality of the approximation is given by the least squares loss function[1]

$$\sigma(X, \Delta) := \frac{\sum_{k=1}^m w_k(\delta_k - d_k(X))^2}{\sum_{k=1}^m w_k \delta_k^2}, \tag{1}$$

Traditionally, this loss function is called *stress* (Kruskal (1964a), Kruskal (1964b)). It is true that in Kruskal's paper there are no weights and the sum of squares of the distances is in the denominator. It is shown in Kruskal and Carroll (1969), and in more detail in de Leeuw (1975), that the solution of the MDS problem for the different denominators are the same, up a scale constant which is irrelevant for MDS.

In definition (1)

- δ_k are m non-negative *pseudo-distances*;
- w_k are m positive *weights*;
- X is an $n \times p$ *configuration*, with coordinates of n *points* in $\mathbb{R}^p$;
- $d_k(X)$ are the (Euclidean) *distances* between the rows of the configuration matrix.

The (unconstrained, least squares, square, symmetric, Euclidean, $p-$dimensional) MDS problem is to minimize σ of (1) over all $n \times p$ configurations and over the set $\Delta \in \mathbb{R}_+^m$ of non-negative pseudo-distances.

The standard iterative methods for Multidimensional Scaling (MDS), such as *smacof* (De Leeuw and Mair (2009), Mair et al. (2022)) and *ALSCAL* (Takane et al. (1977)), can be quite

[1] The symbol := is used for definitions.

slow to converge. Thus there have been various attempts to use standard numerical analysis techniques to accelerate MDS convergence. But none of these methods, despite of being quite successful, has been implemented in the usual MDS techniques.

Ramsay (1975) used multivariate Aitkin acceleration on various problems in psychometrics and statistics. including MDS. In Groenen et al. (1996) standard *smacof* iterations were compared with iterations of the spectral gradient method. Rosman et al. (2008) de Leeuw (2006) de Leeuw (2008)

Lepage-Saucier (2024)

2 MDS Data Structures

Data management and algorithm initialization in *smacofSS()* is handled by R (R Core Team (2025)). We start with an R object of class *dist* containing dissimilarities. Here is a small example of order four.

```
  1 2 3
2 1
3 3 1
4 2 3 1
```

Turn this into MDS data with the utility *makeMDSData()*, which creates an object of class *smacofSSData*.

```
smallData <- makeMDSData(small)
print(smallData)
```

```
$iind
[1] 2 3 4 4 3 4

$jind
[1] 1 2 3 1 1 2

$delta
[1] 1 1 1 2 3 3

$blocks
[1] 3 0 0 1 2 0
```

```
$weights
[1] 1 1 1 1 1 1

$nobj
[1] 4

$ndat
[1] 6

attr(,"class")
[1] "smacofSSData"
```

Note that the data in column *delta* are increasing, and that *blocks* are *tie-blocks*, i.e. they indicate how many elements are equal to the first element of the block. Also, the *weights* are always there in some form or another, even for MDS analyses that are *unweighted* (i.e. when all w_k are equal). An object of class *smacofSSData* is *complete* if all $\binom{n}{2}$ dissimilarities are present. *makeMDSData()* can handle missing data and nontrivial weights. If our example is

```
   1  2  3
2 NA
3  3  1
4 NA  3  1
```

and we add weights, also of class *dist*,

```
  1 2 3
2 1
3 1 3
4 2 1 0
```

then our *smacofSSData* object becomes

```
smallData <- makeMDSData(smallMissing, smallWeights)
print(smallData)
```

```
$iind
[1] 3 3 4

$jind
[1] 2 1 2

$delta
[1] 1 3 3

$blocks
[1] 1 2 0

$weights
[1] 3 1 1

$nobj
[1] 4

$ndat
[1] 3

attr(,"class")
[1] "smacofSSData"
```

In the dist objects *delta* and *weights* the missing data are coded as zero weights, or as dissimilarities and/or weights that are *NA*. Zero dissimilarities do not indicate missing data. Thus weights are always strictly positive and missing data do not enter into the data object at all.

These conventions make it possible to also handle rectangular off-diagonal data, such as this *delta* and *weights*.

```
  1 2 3 4 5 6
2 0
3 0 0
4 0 0 0
5 1 3 1 1
6 2 1 3 3 0
7 3 1 2 3 0 0


  1 2 3 4 5 6
```

```
2 0
3 0 0
4 0 0 0
5 1 1 1 1
6 1 1 1 1 0
7 1 1 1 1 0 0
```

Now *makeMDSData()* gives

```
$iind
 [1] 5 6 7 5 5 6 7 7 5 6 6 7

$jind
 [1] 1 2 2 3 4 1 3 1 2 3 4 4

$delta
 [1] 1 1 1 1 1 2 2 3 3 3 3 3

$blocks
 [1] 5 0 0 0 0 2 0 5 0 0 0 0

$weights
 [1] 1 1 1 1 1 1 1 1 1 1 1 1

$nobj
[1] 7

$ndat
[1] 12

attr(,"class")
[1] "smacofSSData"
```

Note however that handling rectangular data with square symmetric MDS is inefficient, and it is better to use smacof programs specifically designed for rectangular data.

It is of course also possible to construct *smacofSSData* objects in other ways, and to edit the objects generated by *makeMDSData()*, for instance by deleting/adding observations or transforming weights/dissimilarities. As long as the conventions are obeyed that no index pair (i, j) occurs more than once, that the dissimilarities remain sorted, and that the tie-blocks

faithfully reflect ties in the sorted dissimilarities. We do need $i \neq j$, but it is not necessary that always $i > j$.

3 Computation

We minimize stress from (1) by minimizing its numerator $\sum_{k=1}^m w_k(\delta_k - d_k(X))^2$ over the $n \times p$ configurations X and over $\delta \in \Delta \cap \mathcal{S}$, where $\mathcal{S}$ is set of all vectors in $\mathbb{R}^m$ with $\sum_{k=1}^m w_k\delta_k^2 = 1$. In de Leeuw (1975) this is called *explicit normalization* (of the disparities) to contrast it with the *implicit normalization* in (1). It is also shown in de Leeuw (1975) that explicit and implicit normalization give the same solution for δ and X up to a scale constant. See also Bauschke et al. (2018).

The default initial configuration for the iterations is the classical scaling solution, with missing data imputed as average non-missing dissimilarities. Smacof algorithms for MDS use *Alternating Least Squares (ALS)* iterations to minimize stress. ALS is a form of block relaxation applied to least squares loss functions. Specific cases of ALS have been around for a long time. But as a general class of techniques it was introduced in de Leeuw (1968), where it was also given its name. If there are two blocks of variables we alternate finding the optimum for the variables in the first block, with those in the second block fixed at their current values, and finding the optimum for the variables in the second block, with those in the first block fixed at their new current values. In the terminology of Guttman (1968) this strategy defines a *two-phase algorithm*. ALS is especially relevant for non-metric scaling, in which the two sets of unknowns X and δ are nicely separated.

Finding the optimal X for given δ means solving a metric MDS problem. The vector δ is fixed throughout the computations, in other words Δ is a singleton, a set with one element. To solve the optimum X subproblem we use *majorization*, which was introduced to MDS by de Leeuw (1977). The majorization details are now in many places, for example in Borg and Groenen (2005) or Groenen and Van de Velden (2016). Although majorization in metric MDS does not use ALS, it is a form of block relaxation (de Leeuw (1994)) that also proceeds by solving a sequence of relatively simple least squares problems.

In the non-metric case finding the optimum δ for fixed X, and thus fixed $d(X)$, is a *monotone regression* problem, i.e. a linear least squares problem with monotonicity constraints on the parameters. There is a huge literature on monotone regression and the *Pool Adjacent Violators Algorithm (PAVA)* reviewed in De Leeuw et al. (2009). In the MDS context it suffices to refer to Kruskal (1964b).

In our implementation of smacof we alternate one majorization step with one monotone regression step. Thus in the first phase we make only one step towards the conditional minimum over X, while in the second phase we go all the way to the conditional minimum over δ . There

may be some improvement possible by making more majorization steps in the first phase, or by using the over-relaxation step proposed by de Leeuw and Heiser (1980). However, these possible accelerations have not been implemented yet.

In the non-metrix case our algorithm is quite different from that in Kruskal (1964a), Kruskal (1964b). Kruskal defines his non-metric stress as

$$\sigma(X) := \min_{\delta \in \Delta} \frac{\sum_{k=1}^m w_k (\delta_k - d_k(X))^2}{\sum_{k=1}^m w_k d_k^2(X)}. \tag{2}$$

This stress is a function of X only, which is then minimized by a gradient method with a complicated step-size procedure. In Guttman's terminology this is a *one-phase* procedure. Even in the metric case Kruskal's steps differs from smacof's majorization steps, which are gradient steps with a constant step-size that guarantees monotone convergence to a local minimum from any starting point.

In the code used in this paper *smacofSS()* is an R function in the R front end *smacofSS.R* that reads the parameters and data of the problem and then loads a shared library, called *smacofSSEngine.so*, which contains the compiled code of the C routine *smacofSSEngine()*. The C routine *smacofSSEngine()* alternates calls to the compiled C routines *smacofSSMajorize()* and *smacofSSMonotone()*. Because of this modularity the same code can be used with small modifications for alternative MDS methods such as McGee's Elastic Scaling and Sammon Mapping (de Leeuw (2025a), de Leeuw (2025b)).

The github repository has the R and C code, some utilities in R, and a small Makefile for the shared library.

4 Software

4.1 Arguments

The *smacofSS()* function has the following arguments, with default values,

1. *theData*, a *smacofSSData* object.
2. *ndim=2*, dimensionality of MDS analysis.
3. *xinit=NULL*, initial configuration, NULL or an *nobj* by *ndim* matrix.
4. *ties = 1*, ties approach, 1, 2, or 3 for primary, secondary, tertiary.
5. *iord = 2*, acceleration strategy, vector with integers between 0 and 3.
6. *safe = TRUE*, if TRUE guarantees monotone convergence.
7. *itmax = 1000*, maximum number of iterations.
8. *eps = 1e-6*, if successive configurations change less than *eps*, stop.

9. *digits = 10*, digits stress print if verbose is TRUE,
10. *width = 15*, width stress print if verbose is TRUE,
11. *verbose = TRUE*, TRUE/FALSE print stress for each iteration to *stdout*.
12. *weighted = FALSE*, TRUE/FALSE for weighted/unweighted least squares.
13. *ordinal = FALSE*, FALSE for numerical,TRUE for ordinal.

4.2 Value

The list of objects returned by *smacofSS()* mimics, as much as possible, the list returned by the function *smacofSym()* from the smacof package.

1. *delta*, dissimilarities, vector of length m.
2. *dhat*, final pseudo-distances, vector of length m.
3. *confdist*, final distances, vector of length m.
4. *conf* , final configuration, $n \times p$ matrix.
5. *weightmat*, weights, vector of length m.
6. *stress*, final stress.
7. *ndim*, number of dimensions.
8. *init*, initial configuration, $n \times p$ matrix..
9. *niter*, number of iterations.
10. *nobj*, number of objects.
11. *iind*, row indices, vector of length m.
12. *jind*, column indices, vector of length m.
13. *weighted*, was the analysis weighted.
14. *ordinal*, was the analysis ordinal (and if so, which tie approach).
15. *ties*, approach to ties if ordinal (one, two, or three).

One important special case should be kept in mind. In the ordinal case with the primary approach to ties, the data are (potentially) reordered within tie blocks in each iteration. In each iteration we have to reorder the indices *iind*, *jind*, and in the weighted case the weights. There is no need to reorder delta and the blocks, because the only ordering changes are within blocks. Ultimately this means that if smacofSS() returns a list, then in that list *iind*, *jind*, and in the weighted case *weightmat* will be ordered differently from the corresponding columns in the MDS data structure. No reordering is going on using the secondary and tertiary approaches.

5 Details

5.1 Interface C and R

The C routine *smacofSSEngine()* by itself is a complete smacof MDS programs. It would be easy to write a main program in C that do the same job as the R frontend, and compile it to a stand-alone executable.

The C code uses the *.C()* calling conventions and is otherwise not dependent on R in any way. It is important to note that the *.C()* call from R is only executed one time in a *smacofSS* job, when loading the shared library, and after that there is only compiled code running until the end of the job. In other words, everything done in R is either front-end or back-end, and is only done once. Earlier versions of the R/C program had R also managing the iterations. In each iteration there was a *.C()* call to update the configuration and in the ordinal case an additional *.C()* call to update the pseudo-distances. This turned out to be unsatisfactory, because it did not give enough speedup compared to the *smacofSym()* in the *smacof* package on CRAN (De Leeuw and Mair (2009), Mair et al. (2022)).

5.2 Majorization

The majorization updates are of the form $X \leftarrow V^+B(X)X$, where $V$and $B(X)$ are symmetric doubly-centered matrices. Matrix $V$is constant and has off-diagonal elements $-w_{ij}$, matrix $B(X)$ has off-diagonal elements $-w_{ij}\delta_{ij}/d_{ij}(X)$. Matrix V^+ is a generalized inverse of V. For details see, for example, de Leeuw and Heiser (1980) or de Leeuw (1988). If the weights are irreducible, which we can assume without loss of generality (de Leeuw (1977)), then

$$V^+ = (V + \frac{1}{n}ee')^{-1} - \frac{1}{n}ee' \tag{3}$$

can be used to efficiently compute the Moore-Penrose inverse. In the unweighted case we can simply set $V^+ = \frac{1}{n}I$.

If *weighted* is true then smacof requires two multiplications of dense symmetric matrices of order n in each iteration. This is the most expensive part of the calculations. We handle these multiplications in the C routine *smacofSSMajorize()*, which is called by *smacofSSEngine()*, by storing and computing only the part below the diagonal and by using double-centering and symmetry in the calculations. The Moore-Penrose inverse of the weighting matrix $V$is also needed (only once) and it is computed by the C routine *smacofMPInverseV()* using only the lower diagonal elements of $V$with symmetric sweeping.

5.3 Monotone Regression

If *ordinal* is TRUE the monotone regression in each iteration uses the C version of the *monotone()* algorithm of Busing (2022). The *monotone()* algorithm is called in three separate C routines *primaryApproach()*, *secondaryApproach()*, and *tertiaryApproach()*, for which the code is in *smacofIsotone.c*. One of the three tie approaches is called in each iteration by the *smacofSSMonotone()* routine (which in turn is called by *smacofSSEngine()*).

5.4 Initial Configuration

The initial configuration routines are written in R. They are

1. *smacofTorgerson(theData, ndim)*. Returns initial configuration in $nobj \times ndim$ matrix using classical MDS.
2. *smacofGuttman(theData, ndim)*. Returns the Guttman-Lingoes initial configuration in $nobj \times ndim$ matrix.
3. *smacofElegant(theData, ndim)*. Returns the "elegant" initial configuration in $nobj \times ndim$ matrix.

They all need the first *ndim* eigenvalues with the corresponding eigenvalues. We use *eigs-sym()* from the RSpectra package (Qiu and Mei (2024)) for this. We can also generate a random initial configuration with *smacofRandomConfiguration()* in *smacofAuxiliaries.R*. Random configurations are not really useful for data analysis, but can be used in theoretical studies of convergence and local minima.

Each of the three routines is actually a fairly complete metric MDS program that takes a *smacofSSData* object as its argument and returns a value similar in structure to the *smacofSSResult* object returned by *smacofSS()*. Thus the returned object can be used directly the plotting routines of Section 6.2.

In terms of strategy we suggest that in an MDS analysis the researcher first computes one or more initial configurations (which are all independent MDS solutions anyway, except for the random configuration), and then give one or more of these initial configurations as the *xinit* argument to *smacofSS()*.

The initial configuration routines all are based on finding a solution with a good value of *sstress*, defined in the same way as stress, but with squared distances and dissimilarities. Thus sstress is

$$\sigma(X,\Delta) := \frac{\sum_{k=1}^m w_k(\delta_k^2 - d_k^2(X))^2}{\sum_{k=1}^m w_k \delta_k^4}. \tag{4}$$

The *smacofSSResult* returned by the initial configuration routines has an sstress value, instead of a stress value. Of source the stress value of the sulution will be revealed when it is used as an initial configuration in *smacofSS()*. Also, when *smacofSSResult* from an sstress routine is used for a Shepardplot, squared distances and squared dhats are plotted against unsquared deltas so the plot has a quadratic shape.

5.4.1 Torgerson

The default initial configuration is set to NULL for *smacofSS()*. This is mean to emphasize the importance of choosing an initial configuration. But to make life easier for the user if *xinit* is null, then an initial configuration is computed by the *smacofTorgerson()* routine in *smacofAuxiliaries.R*. *smacofTorgerson()* is classical MDS (Torgerson (1958)), with the missing dissimilarities imputed using the average non-missing dissimilarity.

The Torgerson initial configuration works best for complete and unweighted data, but may also be satisfactory for a small number of randomly missing data and for weights that do not vary a great deal. Also remember that in the case of a really bad fit classical MDS may run into the problem of negative eigenvalues. This will give the initial configuration a dimension less than p, and since smacof iterations never increase dimensionality this will mean the final configuration will also have dimension less than p. Thus the Torgerson initial configuration is unsuitable, for example, for full-dimensional scaling (de Leeuw et al. (2016)).

5.4.2 Guttman

The Guttman-Lingoes initial configuration (Guttman (1968)), which handles missing data and unequal weights, is provided as *smacofGuttman()*. It is closely related to Hayashi's Quantification Method IV (Takane (1977)) and unlike the Torgerson method does not have a negative eigenvalue problem.

Write sstress as

$$\sigma(X) = K - 2\sum_{k=1}^{m} w_k \delta_k^2 d_k^2(X) + \sum_{k=1}^{m} w_k d_k^4(X). \tag{5}$$

By homogeneity this is scale-equivalent to maximizing the middle term over all configurations for which the third term is one. The middle term is equal to tr $X'BX$, where B is doubly-centered with off-diagonal elements $-d_{ij}^2(X)$. Note that B is positive semi-definite. The third term $\sum_{k=1}^{m} w_k d_k^4(X)$ is a homogeneous quartic in X, or equivalently a homogeneous quadratic in the elements of $C = XX'$. In the *elegant* algorithm, discussed in Section 5.4.3, we majorize this quadratic to arrive at a simpler unweighted quadratic in C. In the Guttman initial configuration we maximize the middle term using the alternative quartic constraint tr $(X'X)^2 =$ tr $C^2 = 1$. Note that the B takes weights and missing data into account, but the

normalization constraint does not. The stationary equations are $BX = X(X'X)$ and thus the maximum is attained for $X = K\Lambda^{\frac{1}{2}}$, where Λ are the p dominant eigenvalues of B and K are the corresponding eigenvalues.

5.4.3 Elegant

Our most expensive initial configuration routine is *smacofElegant()*, which implements an optimized version of De Leeuw's 1975 "elegant" algorithm, as described recently in de Leeuw (2025c). *smacofElegant()* is a stand-alone iterative metric scaling method, implemented in R. Using *smacofElegant()* means a lot of computation just to obtain an initial configuration for *smacofSS()*. On the other hand it is crucial in MDS to have an initial configuration which is as good as possible, because that is the best guarantee against local minima.

5.4.4 Full-dimensional Scaling

Finally, we have the option to uses the p dominant dimensions of the *full-dimensiponal* scaling solution as initial configuration. We know that full-dimensional scaling converges to the global minimum of stress in $n-1$ dimensions, and actually to the global minimum for all $p \geq r$, where r is the *Gower rank* of the data (de Leeuw (2016)). This will provide a good initial estimate if r is close to p.

5.5 Acceleration

A *iord* is a vector ℓ of $n \geq 1$ integers, with $0 \leq \ell_i \leq 3$. Strategies are used to code n *updates*, with update i has *order* ℓ_i. Iterations repeat strategies cyclically. Thus iord $(0, 0, 3)$ means that iterations use updates of order $(0, 0, 3, 0, 0, 3, ...)$, while $(1, 0)$ means the order sequence $(1, 0, 1, 0, 1, 0, ...)$. The theory in Lepage-Saucier (2024) allows one to go to orders hgher than three, but we will use these in this paper.

To define orders zero to three for smacof we use the Guttman transform Γ of an MDS configuration. In iteration k we compute

$$\Delta_0^{(k)} := X^{(k)}, \tag{6a}$$

$$\Delta_1^{(k)} := \Gamma(X^{(k)}) - X^{(k)}, \tag{6b}$$

$$\Delta_2^{(k)} := \Gamma^2(X^{(k)}) - 2\Gamma(X^{(k)}) + X^{(k)}, \tag{6c}$$

$$\Delta_3^{(k)} := \Gamma^3(X^{(k)}) - 3\Gamma^2(X^{(k)}) + 3\Gamma(X^{(k)}) - X^{(k)}. \tag{6d}$$

0. Order zero are the standard smacof iterations

$$X^{(k+1)} = \Gamma(X^{(k)}). \tag{7}$$

1. For order one

$$\lambda_1^{(k)} := \frac{\langle \Delta_0^{(k)}, \Delta_1^{(k)} \rangle}{\langle \Delta_1^{(k)}, \Delta_1^{(k)} \rangle}, \tag{8a}$$

$$X^{(k+1)} = \Delta_0^{(k)} + \lambda_1^{(k)} \Delta_1^{(k)}. \tag{8b}$$

2. For order two

$$\lambda_2^{(k)} := \frac{\langle \Delta_1^{(k)}, \Delta_2^{(k)} \rangle}{\langle \Delta_2^{(k)}, \Delta_2^{(k)} \rangle}, \tag{9a}$$

$$X^{(k+1)} = \Delta_0^{(k)} + 2\lambda_2^{(k)} \Delta_1^{(k)} + \left[\lambda_2^{(k)}\right]^2 \Delta_2^{(k)}. \tag{9b}$$

3. For order three

$$\lambda_3^{(k)} := \frac{\langle \Delta_2^{(k)}, \Delta_3^{(k)} \rangle}{\langle \Delta_3^{(k)}, \Delta_3^{(k)} \rangle}, \tag{10a}$$

$$X^{(k+1)} = \Delta_0^{(k)} + 3\lambda_3^{(k)} \Delta_1^{(k)} + 3\left[\lambda_3^{(k)}\right]^2 \Delta_2^{(k)} + \left[\lambda_3^{(k)}\right]^3 \Delta_3^{(k)}. \tag{10b}$$

6 Utilities

6.1 Data Utilities

There are four functions in smacofDataUtilities.R

1. *makeMDSData(delta, weights = NULL)*. Returns *smacofSSData* object from dist objects.
2. *fromMDSData(theData)*. Returns *dist* objects of dissimilarities and weights from *smacofSSData* object.
3. *matrixPrint(x, digits = 6, width = 8, format = "f", flag = "+")*. Formats and prints a matrix.
4. *smacofRandomConfiguration(theData, ndim = 2)*. Random initial configuration.

6.2 Plotting

There are three plot routines in the file *smacofPlots.R*.

1. *smacofShepardPlot(h, main = "ShepardPlot", fitlines = TRUE, colline = "RED", colpoint = "BLUE", resolution = 100, lwd = 2, cex = 1, pch = 16)*
2. *smacofConfigurationPlot(h, main = "ConfigurationPlot", labels = NULL, dim1 = 1, dim2 = 2, pch = 16, col = "RED", cex = 1)*
3. *smacofDistDhatPlot(h, fitlines = TRUE, colline = "RED", colpoint = "BLUE", main = "Dist-Dhat Plot", cex = 1, lwd = 2, pch = 16)*

The Shepard plot has the original dissimilarities on the horizontal axis, and both the pseudo-distances and the distances on the vertical axis. Pseudo-distances are plotted as points of color *colline*, and are connected by a line of color *colline*. Distances are plotted as points if color *colpoint*. If *fitlines* is TRUE then black vertical lines from the distance point to the pseudo-distance point are drawn.

The configuration plot plots two dimensions *dim1* and *dim2* of the configuration. If *labels* is NULL points are drawn with symbol *pch*, otherwise a vector of labels is used.

A DistDhat-plot has distances on the horizontal axis and pseudo-distances on the vertical axes. Points are drawn using symbol *pch* in color *colpoint*. The line through the origin with slope one is drawn in color *colline*. If *fitlines* is true then black lines from the points to their orthogonal projections on the line are drawn.

Residual plot

6.3 Newton

7 Example Data Sets

There are a number of example data sets in the directory smacofSSData. They are mostly of the 20th century boomer type: small and collected from the aggregated judgments of human subjects. All data are available both as dist objects and as MDS data structures. The Wish and Iris data are not used in our computations but are included as examples of MDS data structures.

7.1 Ekman Data

Similarity data between 14 colors from Ekman (1954). Rating scale similarity judgments averaged over subjects, linearly converted to unit interval dissimilarities.

7.2 Morse Data

Dissimilarity between the Morse codes of 36 letters and numbers from Rothkopf (1957). Confusion probabilities transformed to dissimilarities.

7.3 Gruijter Data

Dissimilarities of nine Dutch political parties in 1967, from De Gruijter (1967). Collected using the method of triads and averaged over 100 subjects.

7.4 Wish Data

Rating scale similarities between 12 nations, collected by Wish (1971). Averaged over subjects, subtracted from maximum scale value to form dissimilarities.

7.5 Iris Data

Classical iris data from Anderson (1936) via Fisher (1936). Euclidean distances over four measurements on 150 irises.

8 Comparisons

8.1 Initial

In this section we use the Ekman and Gruijter data to compare the three different initial configurations. Parameters are all set at defaults, which means a two-dimensional solution with unweighted and numerical options.

We start with the Ekman data, which have a very good fit in two dimensions. From the Torgerson initial configuration we need 7 iterations to converge to stress 0.0172132. From the

Guttman initial configuration stress is 0.0172132 after 6 iterations. And from elegant we use 6 iterations to arrive at 0.0172132.

We repeat this for the Gruijter data, which have a poor fit in two dimensions. From the Torgerson initial configuration we need 21 iterations to converge to stress 0.0267299. From the Guttman initial configuration stress is 0.0267299 after 16 iterations. And from elegant we use 22 iterations to arrive at 0.0187831.

For both Ekman and Gruijter it seems that smacofSS() arrives at the same solution, no matter which initial configuration we use. There is some small difference in the number of iterations that are required, and it seems the Torgerson initial configuration holds its own. *smacofElegant()* is expensive and does not seem to deliver any extra efficiency. So, on the basis of the limited experience reported here, we do not recommend it for general use.

8.2 Outcome

In this section we compare the output (final stress, number of iterations) of smacofSym() and smacofSS() using the Ekman and Morse data. Note that smacofSym() reports the square root of the final stress, following Kruskal (1964a), so for comparison purposes we square it again. All runs are started with the two-dimensional Torgerson solution and have the stop criteria *eps* equal to 10^{-10} and *itmax* equal to 1000 (except when using the tertiary approach when *itmax* is 10000). In the weighted case the weights for Ekman are $w_k = \delta_k^2$, for Morse they are $w_k = \delta_k^{-1}$.

Table 1: Comparison smacofSym() and smacofSS() results Ekman data

	Sym stress	Sym niter	SS stress	SS niter
unweighted numerical	0.5067831	198	0.0172132	12
unweighted ordinal ties = 1	0.1221884	375	0.0005337	101
unweighted ordinal ties = 2	0.1280550	290	0.0009977	86
unweighted ordinal ties = 3	0.0077821	403	0.0000000	5965
weighted numerical	0.0361753	1000	0.0172132	13
weighted ordinal ties = 1	0.0124109	848	0.0005337	102
weighted ordinal ties = 2	0.0122130	1000	0.0009977	86
weighted ordinal ties = 3	0.0034770	490	0.0000000	5965

Table 2: Comparison smacofSym() and smacofSS() results Morse data

	Sym stress	Sym niter	SS stress	SS niter
unweighted numerical	0.0899492	238	0.0899492	40
unweighted ordinal ties = 1	0.0326557	143	0.0326557	75
unweighted ordinal ties = 2	0.0406405	135	0.0406405	69
unweighted ordinal ties = 3	0.0000018	351	0.0000003	1000
weighted numerical	0.0977124	317	0.0899492	39
weighted ordinal ties = 1	0.0346208	117	0.0326557	74
weighted ordinal ties = 2	0.0425777	99	0.0406405	71
weighted ordinal ties = 3	0.0000025	289	0.0000003	1000

The conclusion is clear. For both the Ekman and the Morse example the results of *smacofSym()* and *smacofSS()* are identical. Not only the final stress (and thus presumably the final configuration) but also the number of iterations (and thus presumably all iterations). This boosts our confidence in the correctness of both programs.

8.3 Time

The *microbenchmark* package (Mersmann (2024)) is mainly intended to time small pieces of code, not complete programs. Nevertheless we will use it to compare *smacofSym()* and *smacofSS()*, until something more appropriate (and equally easy to use) comes along. We again use the Ekman and Morse data with default options. From the microbenchmark output we find the median time over 100 runs each of the two programs.

Table 3: Comparison smacofSym() and smacofSS() running times Ekman Data

	SS time	Sym time	ratio Sym/SS
unweighted numerical	172364.0	13924932	80.787940
unweighted ordinal ties = 1	797142.5	11881042	14.904539
unweighted ordinal ties = 2	234192.0	10036513	42.855917
unweighted ordinal ties = 3	885928.0	7436190	8.393674
weighted numerical	188395.0	33761552	179.206202
weighted ordinal ties = 1	876047.0	53566869	61.146113
weighted ordinal ties = 2	301801.0	25726250	85.242426
weighted ordinal ties = 3	1086151.5	45658892	42.037314

The entries in the table for the tertiary approach are suspect, because tertiary uses a huge number of iterations to arrive at a perfect solution. In some of the microbenchmark runs it may actually use the full 10000 iterations. From the better conditioned numerical and ordinal solutions we see that *smacofSS()* is between 10 and 55 times as fast as *smacofSym()*.

Table 4: Comparison smacofSym() and smacofSS() running times Morse data

	SS time	Sym time	ratio Sym/SS
unweighted numerical	575045.5	13514564	23.501729
unweighted ordinal ties = 1	1595248.5	13668170	8.568051
unweighted ordinal ties = 2	814506.0	7036030	8.638402
unweighted ordinal ties = 3	7182626.0	20621524	2.871028
weighted numerical	639415.5	17259258	26.992241
weighted ordinal ties = 1	1679852.0	10715104	6.378600
weighted ordinal ties = 2	948719.5	5291808	5.577843
weighted ordinal ties = 3	8865676.0	17507922	1.974798

We see that for the Morse data *smacofSS()* is 3 to 15 times faster than *smacofSym()*. The difference is largest in the numerical case, and it is also larger for unweighted than for weighted. Strangely enough for the ordinal options the weighted case is faster than the corresponding non-weighted case, although a majorization iteration in the weighted case is more expensive. This is undoubtedly because the outcome results in the previous section show that fewer iterations are used in the weighted case. We also see that secondary is faster than primary and tertiary (which is rather well-behaved in the Morse example) is faster than secondary.

We see that (in these examples) the ratio of execution times for *smacofSym()* and *smacofSS()* is larger for the smaller example. We verify this for the Iris example, which has 11175 dissimilarities. For unweighted/numerical the Sym/SS ratio is 2.5, for unweighted/ordinal it is 1.06. Execution times are influenced by many factors. In the majorization step *smacofSym()* uses full matrices, which is wasteful in terms of storage, but allows access to the efficient matrix multiplication routines in R (which are already in compiled C code). *smacofSS()* has the more flexible *smacofSSData* structure, which requires less storage but more index manipulation. Both *smacofSym()* and *smacofSS()* use efficient monotone regression routines in C, so the more time the programs spend in the monotone regression step the more equal execution times will be. A reasonable hypothesis is that using *smacofSSData* will be relatively advantageous if there are many missing data and if there are weights. But more precise profiling will be a worthwhile future project.

The two examples are very different because Ekman has an excellent fit, while Morse has a much poorer one. In both examples the tertiary approach to ties leads to a perfect fit in Ekman and a near perfect fit in Morse. Morse (630 dissimilarities) is quite a bit bigger than Ekman (91

dissimilarities) and Morse has more ties (68 tie-blocks of average size about 9) than Ekman (47 tie-blocks of average size about 2). Except for tertiary Morse requires considerably more iterations than Ekman.

Note, by the way, that we have used microbenchmark in such a way that smacofSS() is always run after the corresponding smacofSym(), and that the eight different basic combinations or weighted and ordinal are always run in the same order.

9 Comparing Strategies

In this section we compare the strategies (x) and $(0, x)$, where x is one of 0,1,2,3. Of course iord (0) is identical to iord $(0, 0)$, because both implement a sequence of ordinary smacof iterations. In addition we vary the *safe* parameter.

9.1 Data

9.2 Number of Iterations

We iterate a maximum of 1000 times and we stop if the sup-norm $\|X^{(k+1)} - X^{(k)}\|_\infty$ is less than 10^{-6}. This is a much stricter stop citerium as the one used in *smacof*, which stops if the change in loss function value $\sigma^{(k)} - \sigma^{(k+1)}$ is less than 10^{-6}. We always start with the classical scaling (Torgerson-Gower) solution.

9.2.1 Strategy (x)

Table 5: iord (x), safe = FALSE

	(0)	(1)	(2)	(3)
ekmanData	27	15	7	6
gruijterData	208	116	21	24
morseData	202	114	28	23

Table 6: iord (x), safe = TRUE

	(0)	(1)	(2)	(3)
ekmanData	27	15	7	6
gruijterData	208	116	21	19
morseData	202	114	29	24

9.2.2 Strategy (0,x)

Table 7: iord (0,x), safe = FALSE

	(0,0)	(0,1)	(0,2)	(0,3)
ekmanData	27	17	9	7
gruijterData	208	139	25	27
morseData	202	135	31	31

Table 8: iord (0,x), safe = TRUE

	(0,0)	(0,1)	(0,2)	(0,3)
ekmanData	27	17	9	7
gruijterData	208	139	25	27
morseData	202	135	31	27

9.3 Timing

9.3.1 Strategy (x)

Table 9: iord (x), safe = FALSE, language = R

	(0)	(1)	(2)	(3)
ekmanData	1964	57315	537	532
gruijterData	32028	27371	2218	3527
morseData	33178	56410	3547	4445

Table 10: iord (x), safe = FALSE, language = C

	(0)	(1)	(2)	(3)
ekmanData	270	834	258	258
gruijterData	207	481	145	151
morseData	1886	3821	1498	1515

Table 11: iord (x), safe = TRUE, language = R

	(0)	(1)	(2)	(3)
ekmanData	1974	1080	535	526
gruijterData	32564	24020	2089	2575
morseData	32841	23130	3222	4190

Table 12: iord (x), safe = TRUE, language = C

	(0)	(1)	(2)	(3)
ekmanData	283	273	266	266
gruijterData	228	192	156	157
morseData	2187	1847	1552	1554

9.3.2 Strategy (0,x)

Table 13: iord (0,x), safe = FALSE, language = R

	(0,1)	(0,2)	(0,3)
ekmanData	1467	584	656
gruijterData	26266	2910	2543
morseData	26074	4162	4028

Table 14: iord (0,x), safe = FALSE, language = C

	(0,1)	(0,2)	(0,3)
ekmanData	275	270	270

	(0,1)	(0,2)	(0,3)
gruijterData	192	156	158
morseData	1730	1496	1521

Table 15: iord (0,x), safe = TRUE, language = R

	(0,1)	(0,2)	(0,3)
ekmanData	1466	584	654
gruijterData	25584	2544	2795
morseData	25898	4103	3986

Table 16: iord (0,x), safe = TRUE, language = C

	(0,1)	(0,2)	(0,3)
ekmanData	268	264	263
gruijterData	190	145	149
morseData	1936	1549	1542

9.4 Nonmetric

Table 17: Nonmetric, iord (x), safe = TRUE

	(0)	(1)	(2)	(3)
ekmanData	99	33	41	42
gruijterData	151	82	80	76
morseData	134	72	41	38

Table 18: Nonmetric, iord (x), safe = TRUE

	(0)	(1)	(2)	(3)
ekmanData	1071	532	604	629
gruijterData	962	586	584	573
morseData	4022	2842	2298	2297

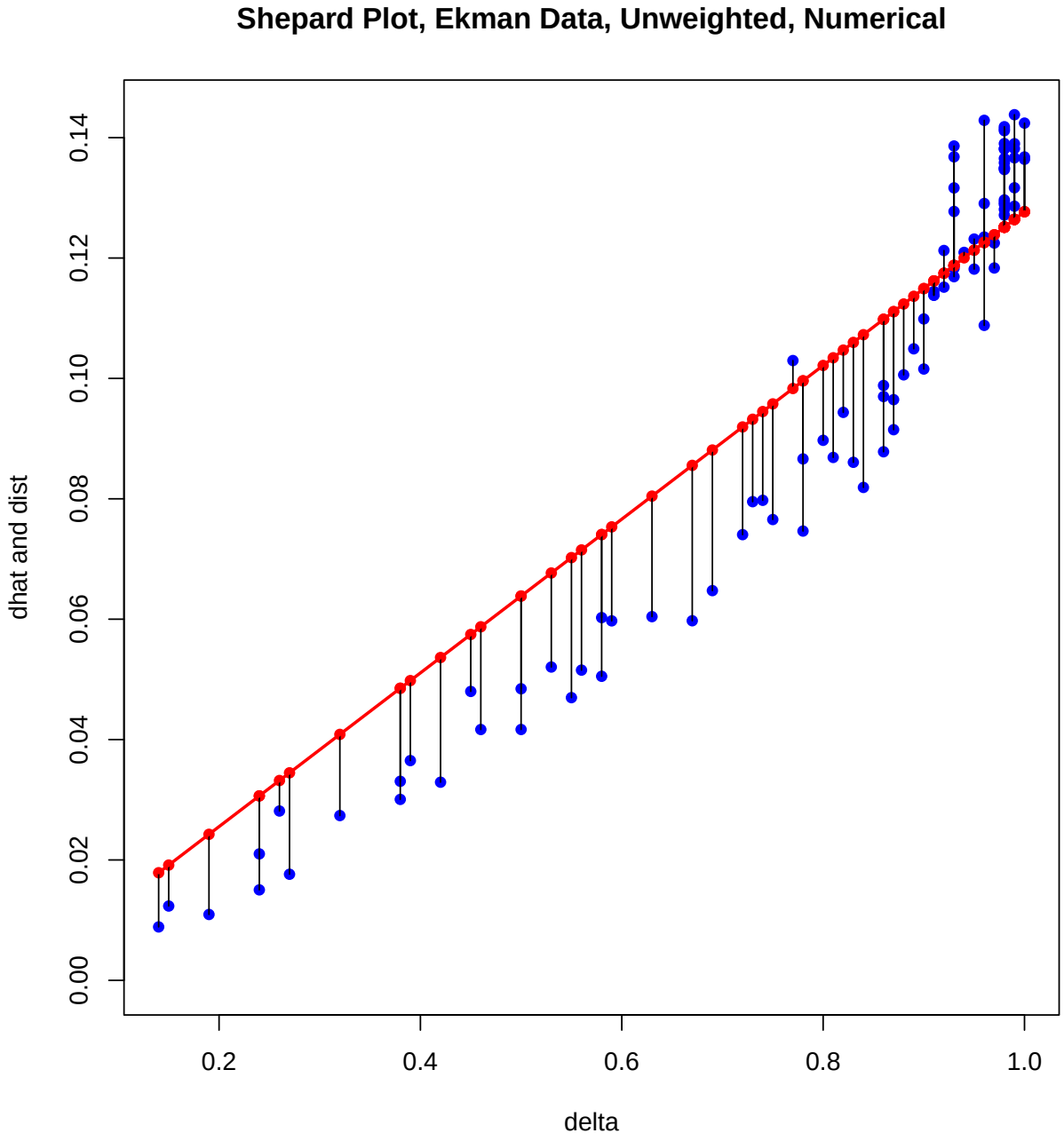
10 Plots

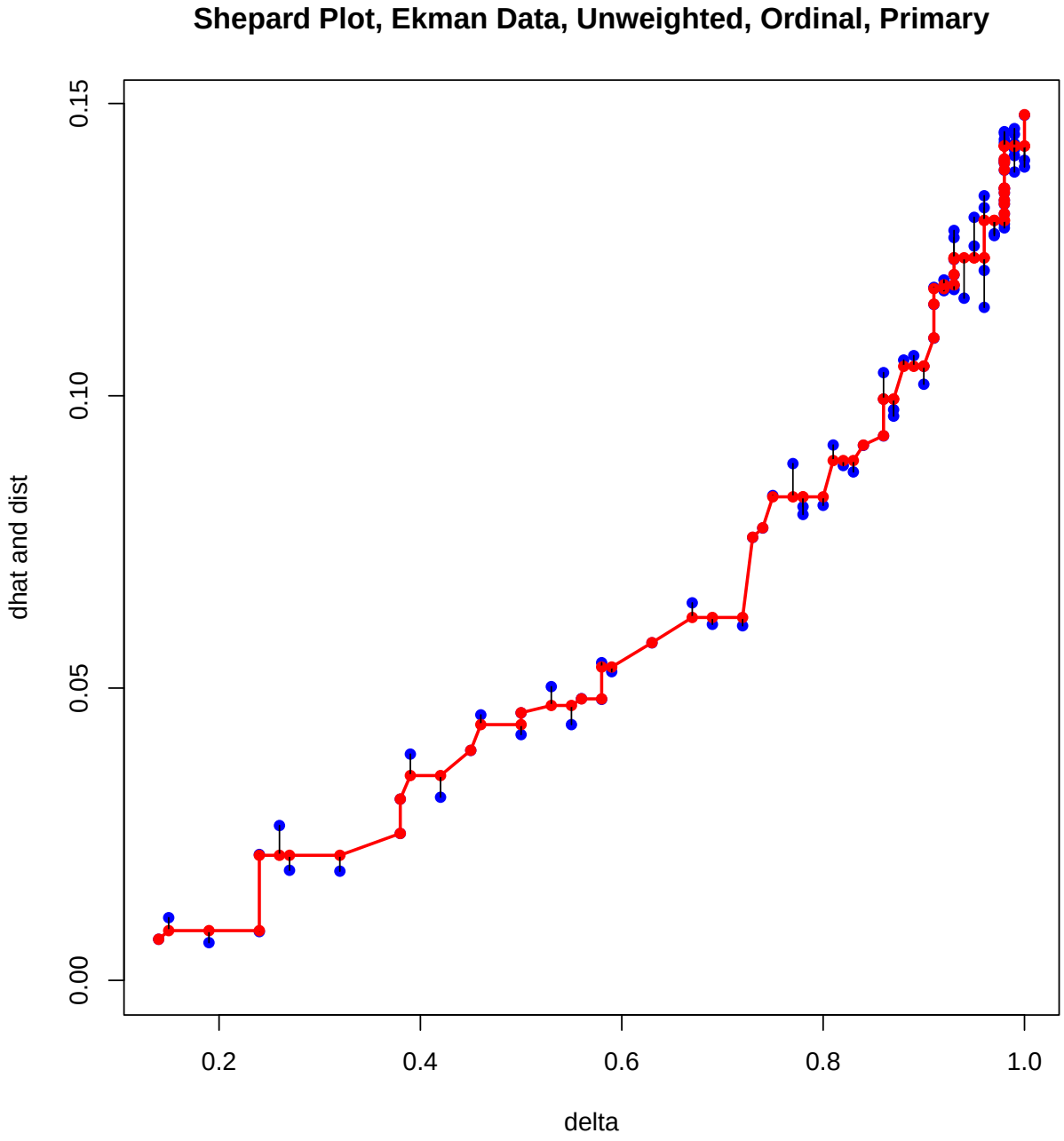
To show some plots we repeat eight possible analyses (numeric/ordinal, where ordinal has three tie approaches, and weighted/unweighted) using the Ekman and Morse data. The number of iterations and the final stress have already been reported in the comparisons section. Both data sets have a fairly large number of ties, so we expect the choice of the ties approach to have some effect.

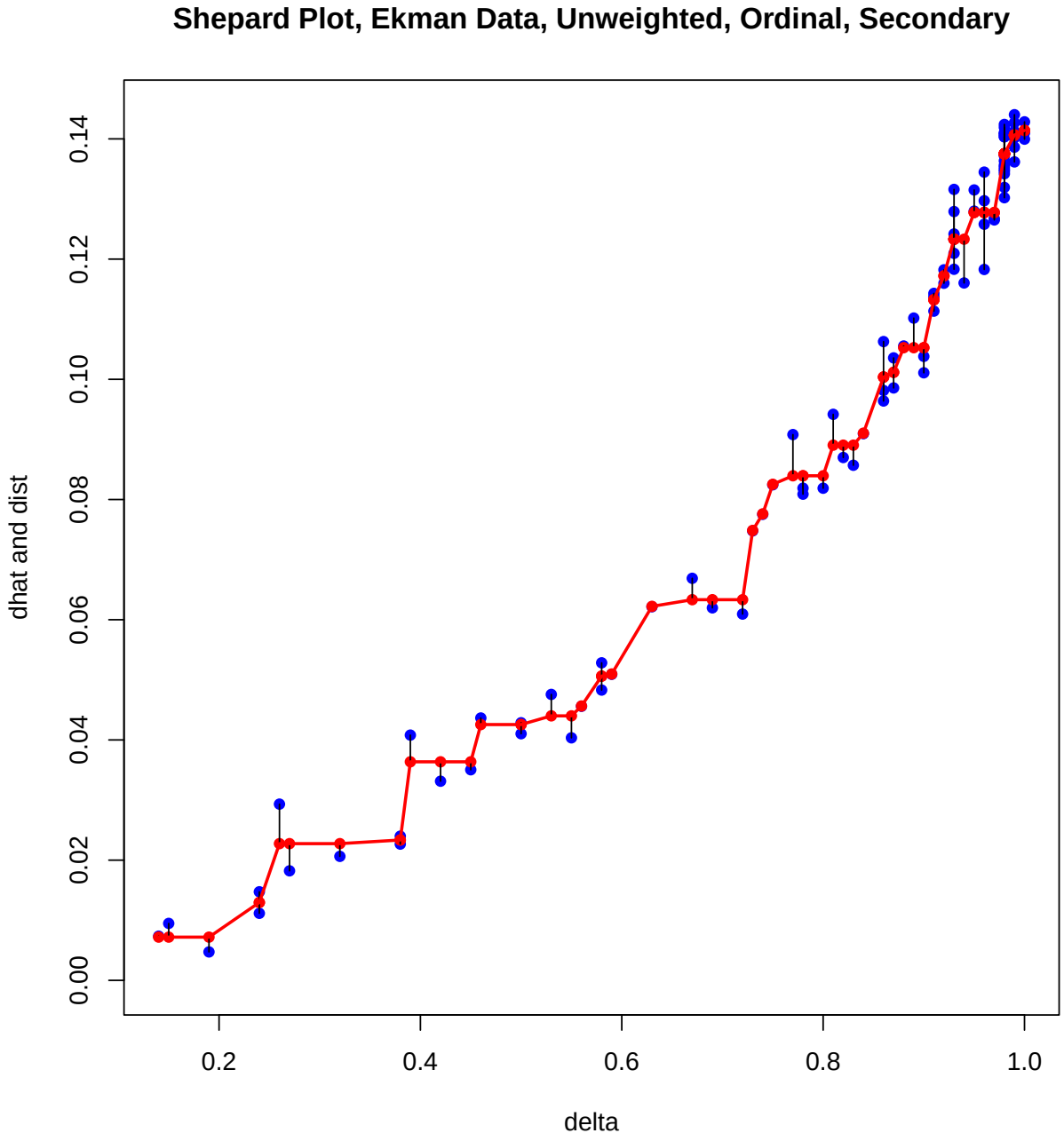
The next eight pages have Shepard plots and DistDhat plots for each of the four unweighted analyses. No plots are given for the weighted analyses, because they would illustrate essentially the same points. We made the plots large so they show some detail. Shepard plots have *fitlines* equal to TRUE, DistDhat plots have *fitlines* equal to TRUE for the Ekman data and FALSE for the Morse data. The sum of the squared lengths of the vertical fitlines in the Shepard plots is the stress. In the DistDhat plots the sum of squares of distances of the points to their orthogonal projections on the line is also the stress. Thus we can see from the plots where the largest residuals are, although the plot does not show which pair of points the fitlines correspond with. That information can easily be obtained from the numerical output.

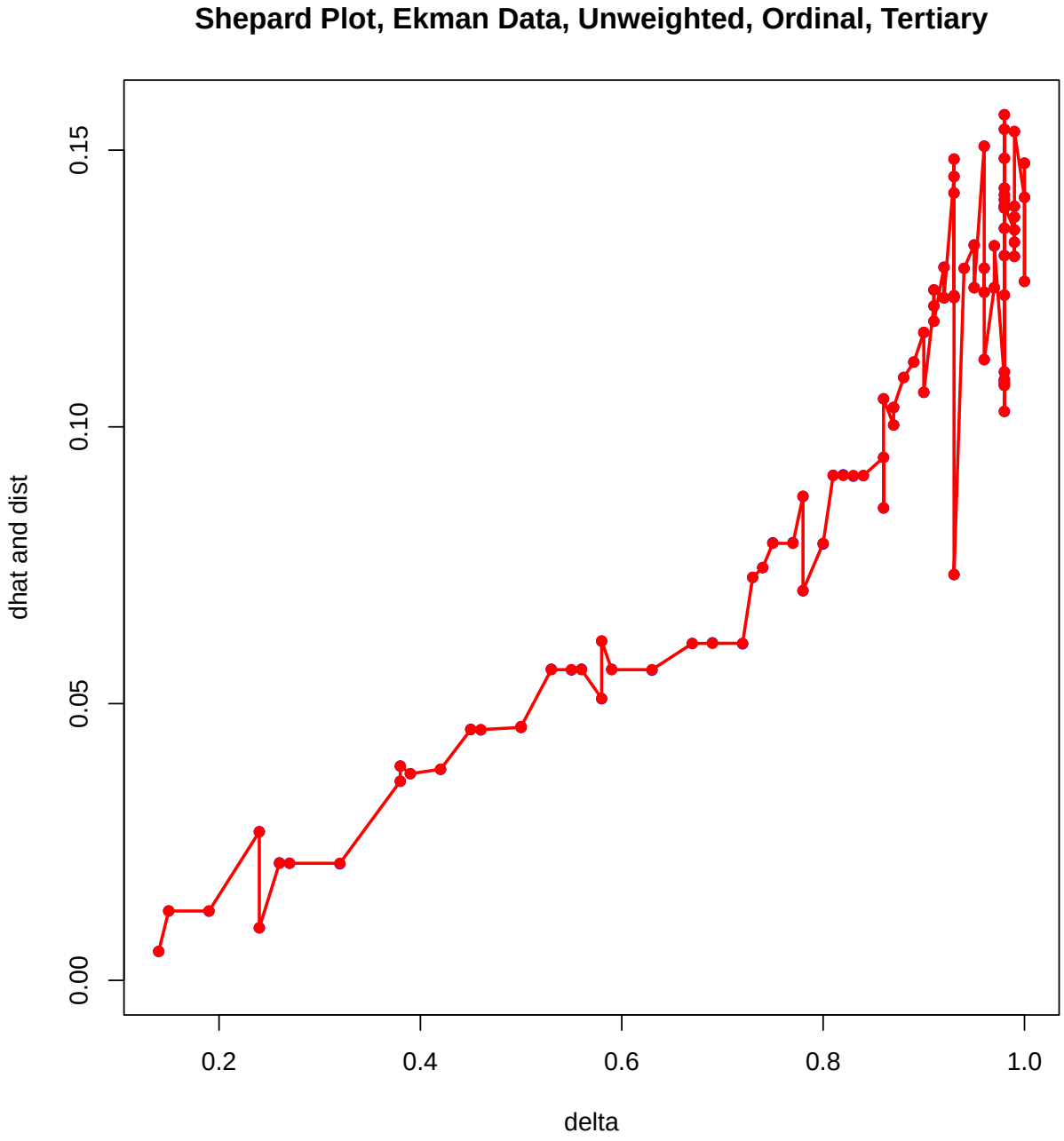
The Ekman example has an exceptionally good fit, even in the numerical case. Still, allowing for ordinal transformations gives a major improvement. Especially the DistDhat plots in the ordinal case show the different ways of handling tie blocks. The tertiary approach gives what is essentially a perfect fit, but the Shepard plot shows that in order to achieve this deviations from monotonicity are required.

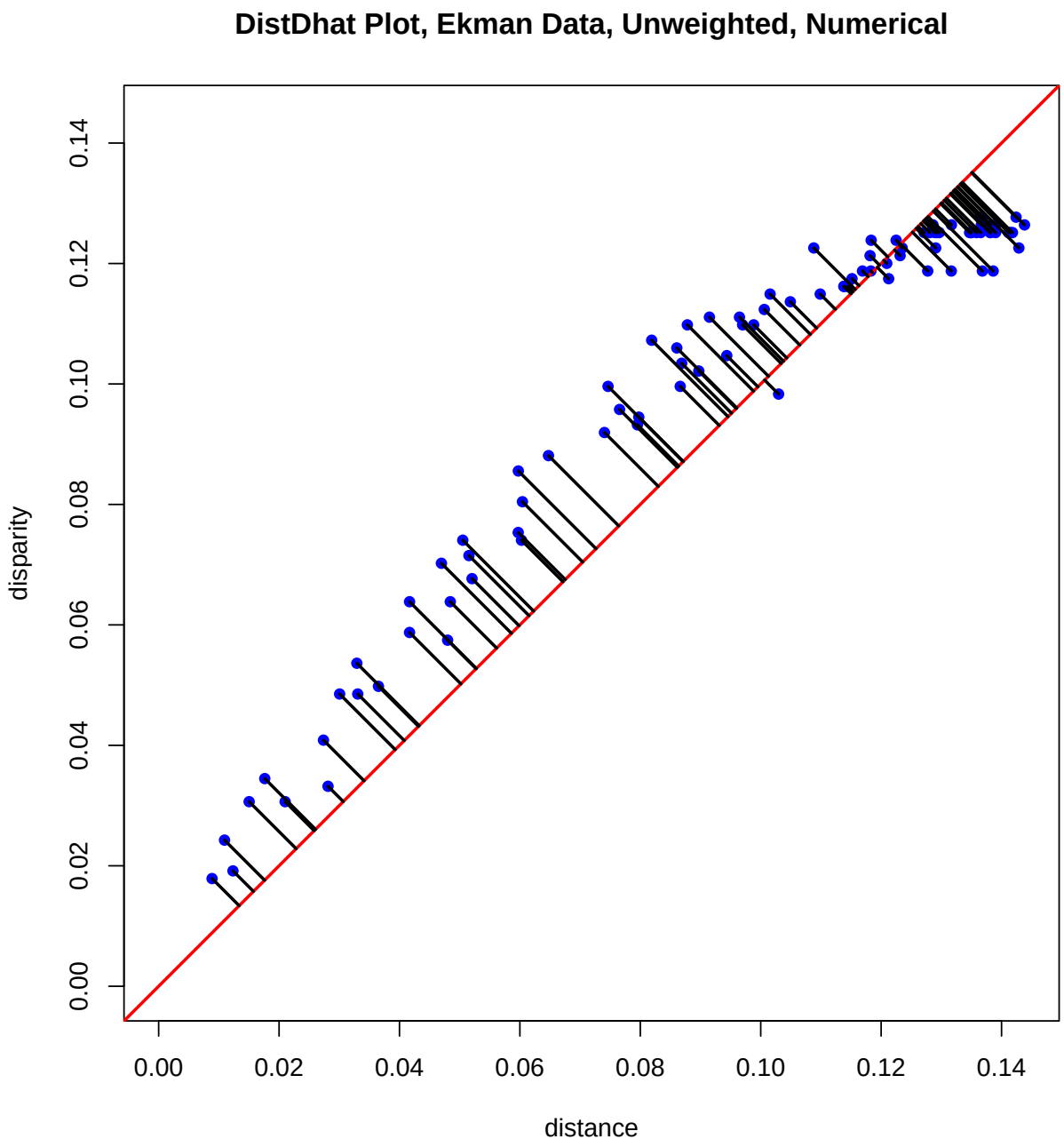
The Morse example has a bad numerical fit, and the improvements by the ordinal options are huge. There are no fitlines in the DistDhat plots, because that would mainly result in big black blobs. As in the Ekman example the tertiary approach gives close to perfect fit, at the cost of many deviations from monotonicity.

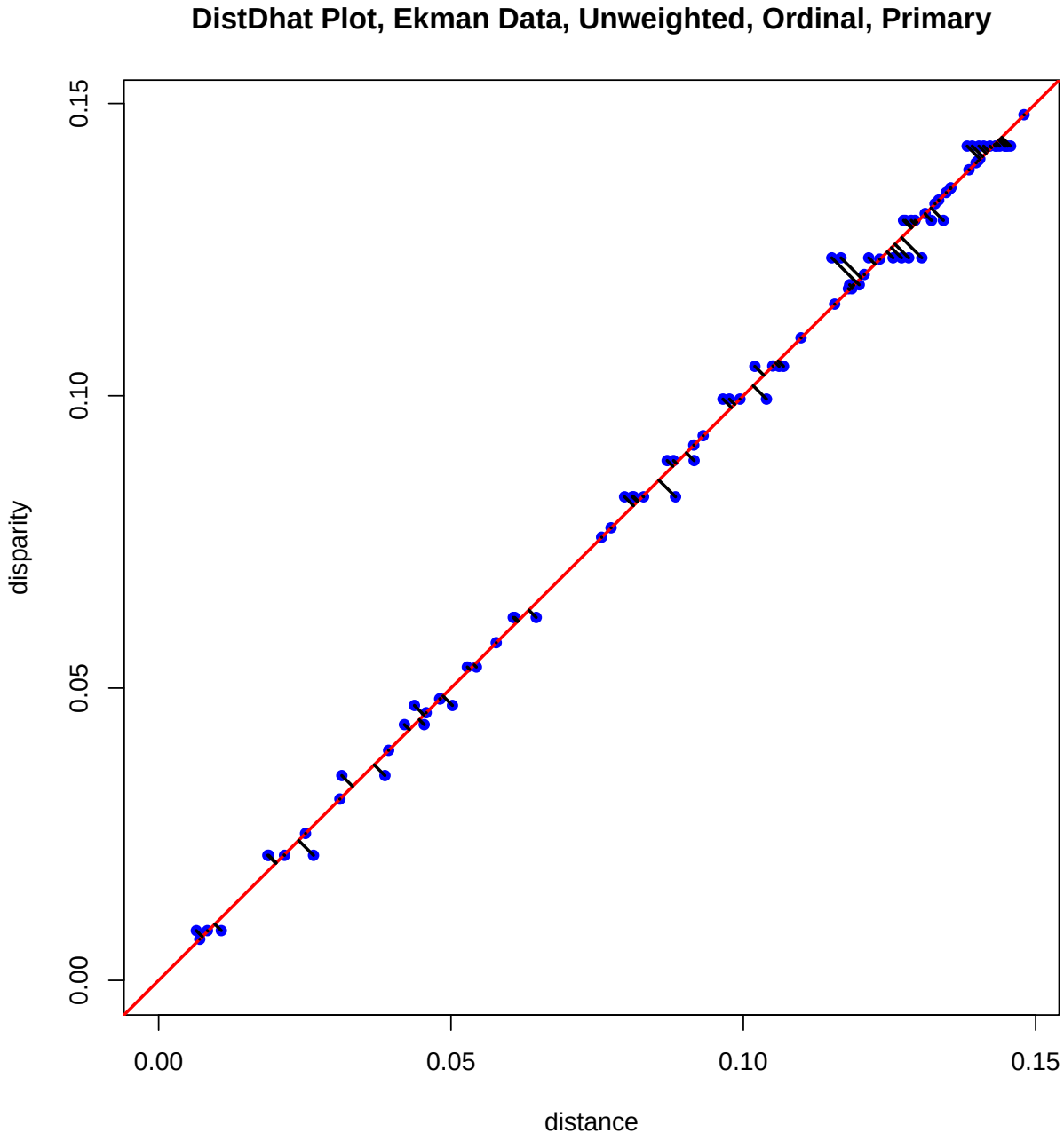

Shepard Plot, Ekman Data, Unweighted, Numerical
dhat and dist
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
0.2
0.4
0.6
0.8
1.0
delta

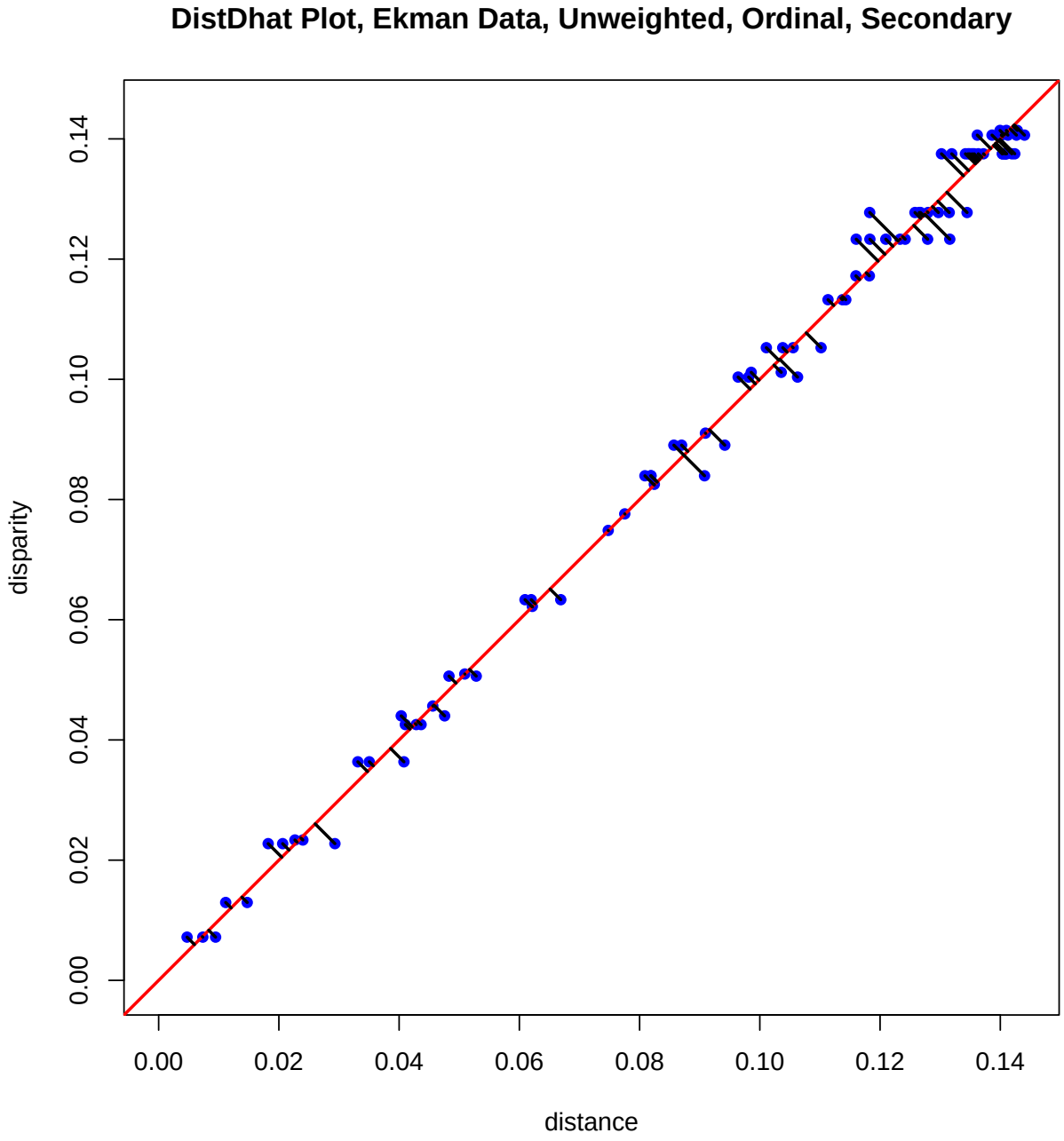

Shepard Plot, Ekman Data, Unweighted, Ordinal, Primary
dhat and dist
0.00
0.05
0.10
0.15
0.2
0.4
0.6
0.8
1.0
delta

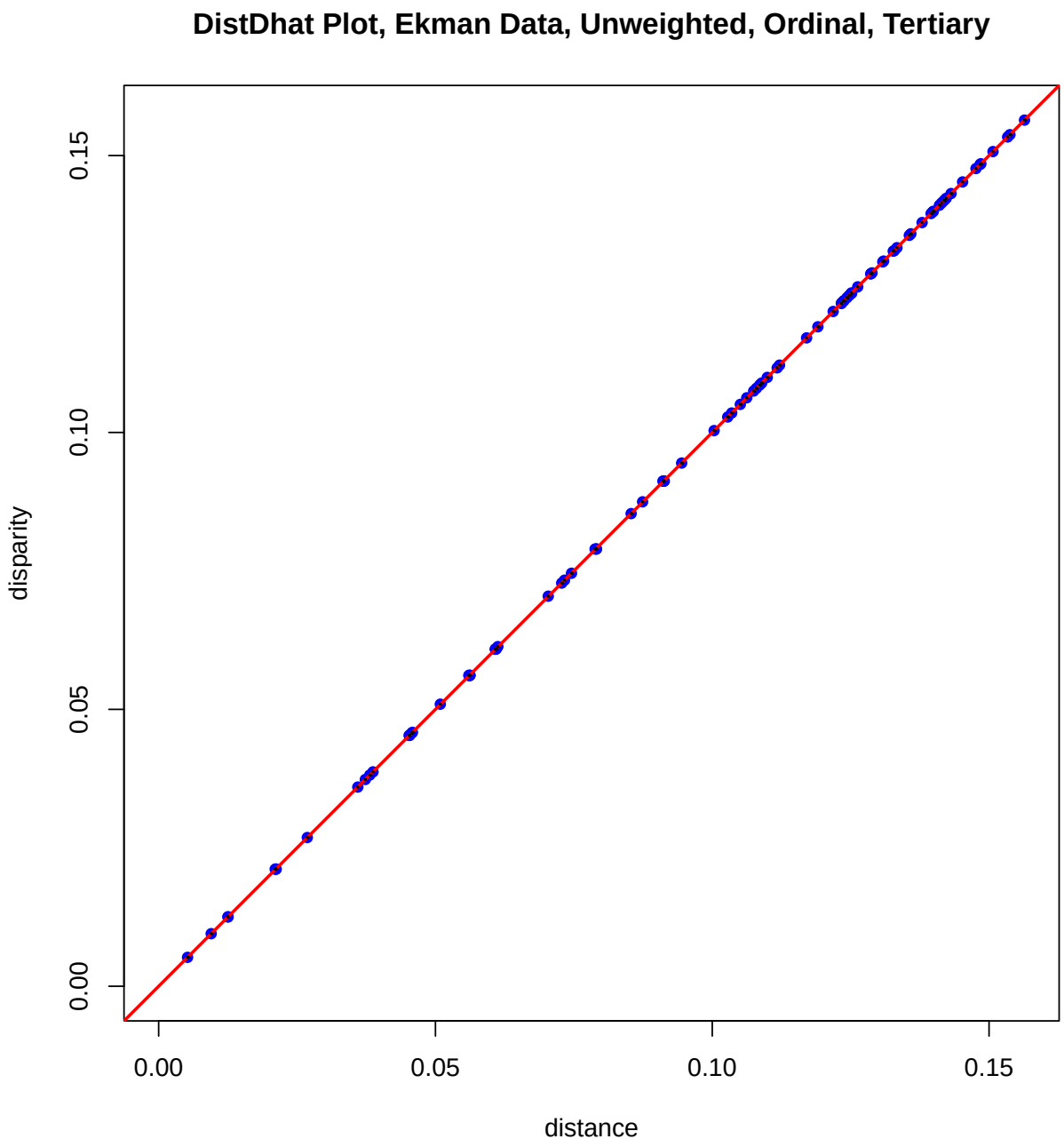

Shepard Plot, Ekman Data, Unweighted, Ordinal, Secondary
dhat and dist
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
0.2
0.4
0.6
0.8
1.0
delta

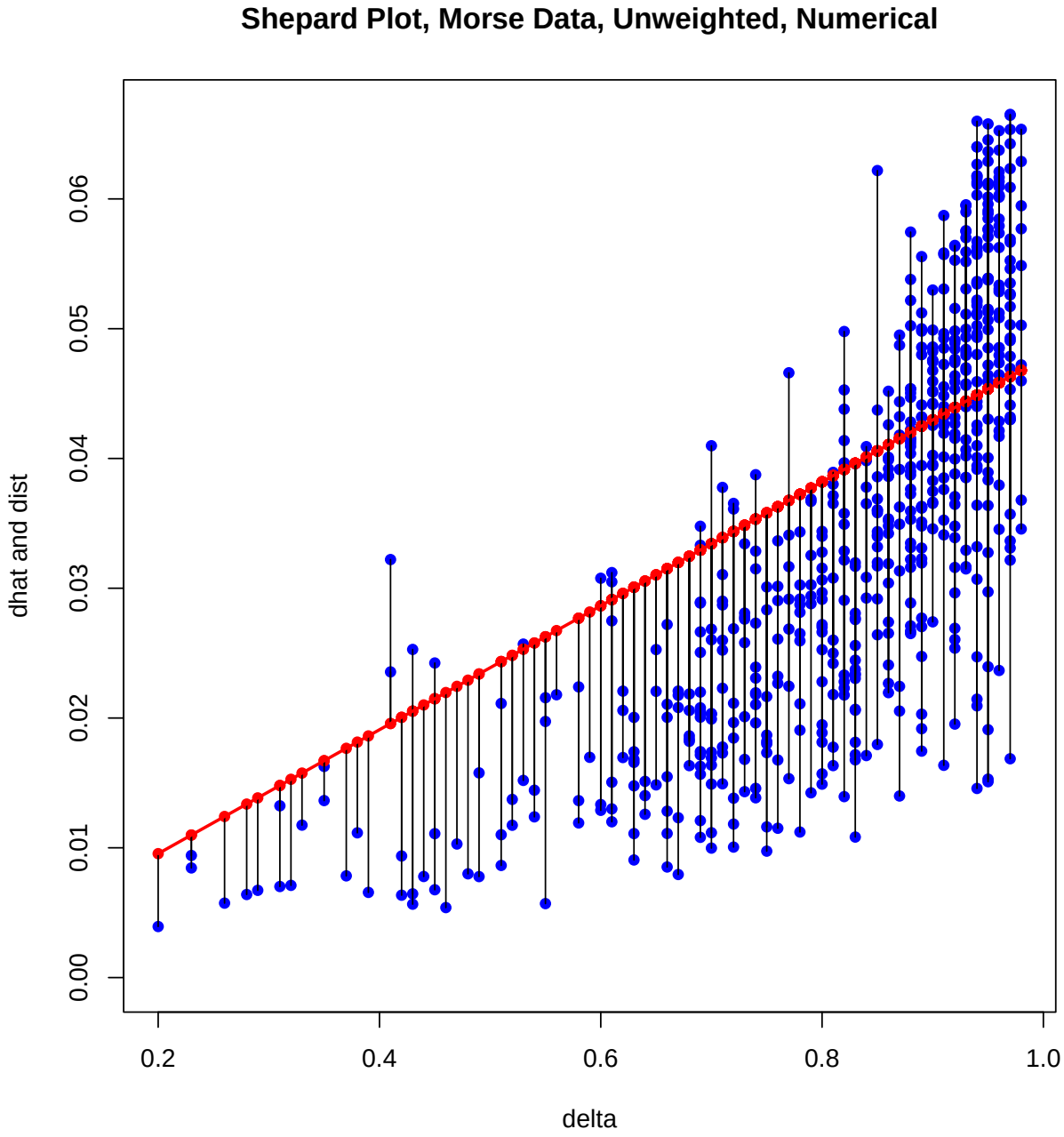

Shepard Plot, Ekman Data, Unweighted, Ordinal, Tertiary
dhat and dist
0.00
0.05
0.10
0.15
0.2
0.4
0.6
0.8
1.0
delta

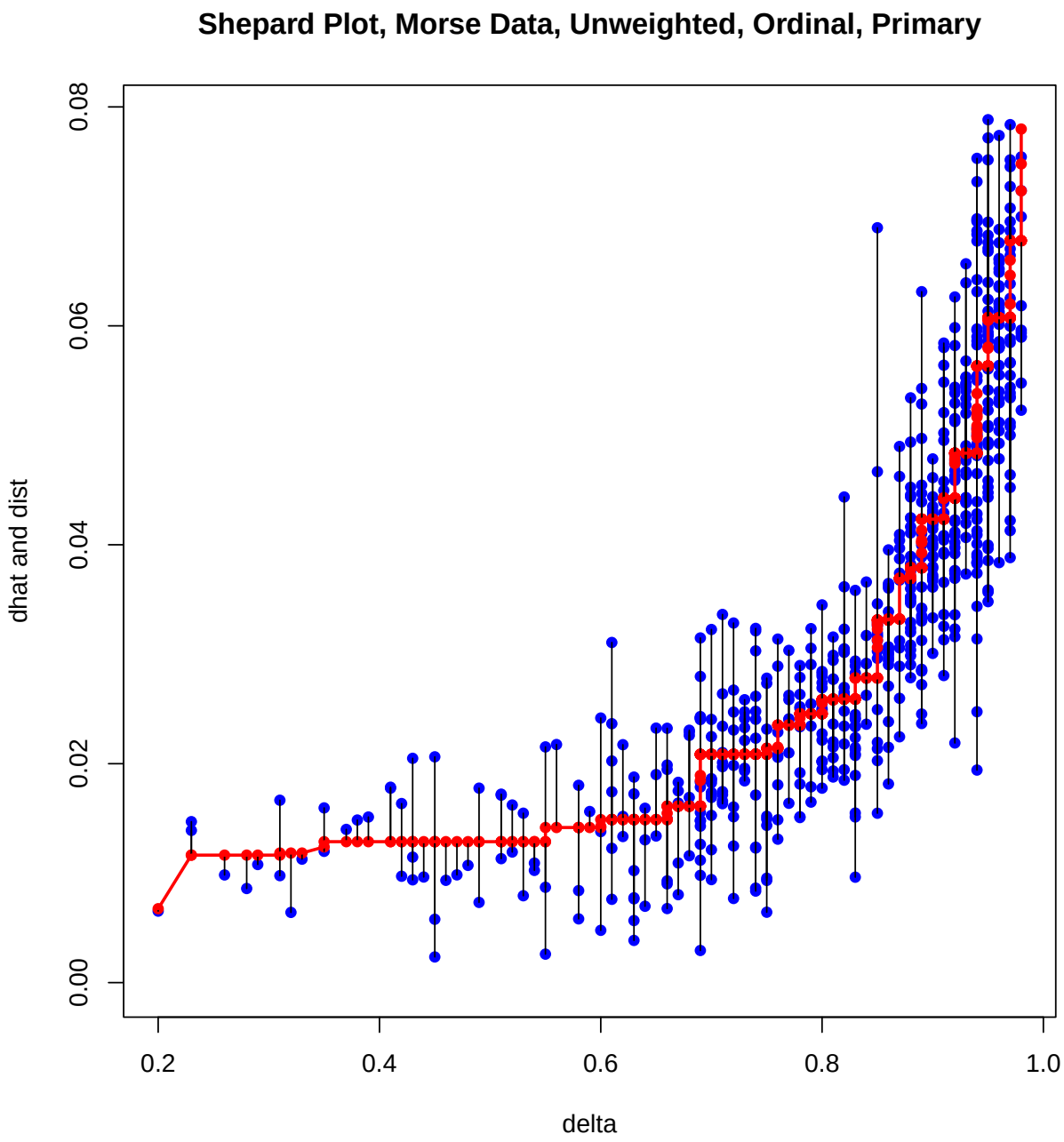

DistDhat Plot, Ekman Data, Unweighted, Numerical
disparity
distance
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14

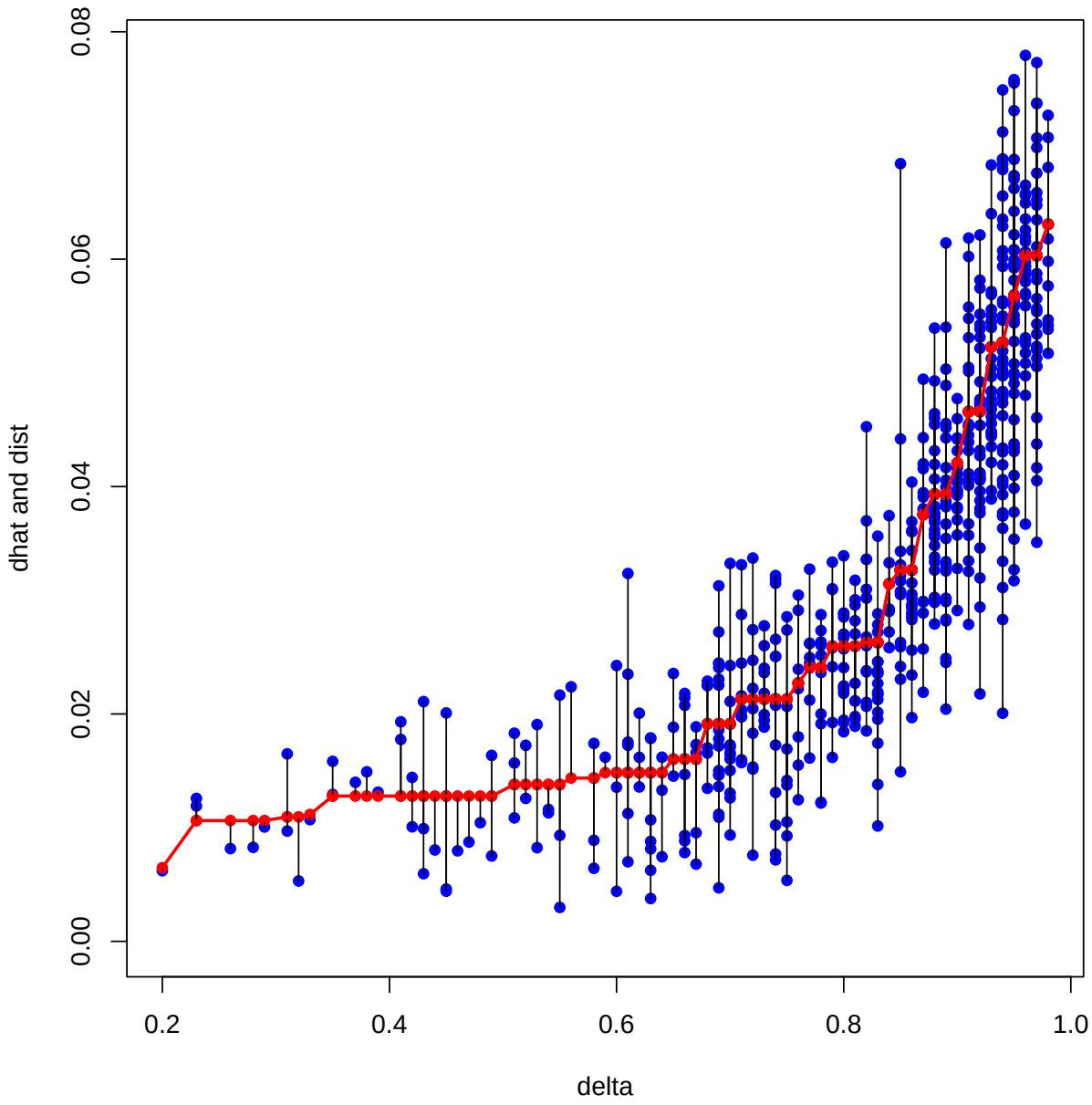

DistDhat Plot, Ekman Data, Unweighted, Ordinal, Primary
disparity
distance
0.00
0.05
0.10
0.15

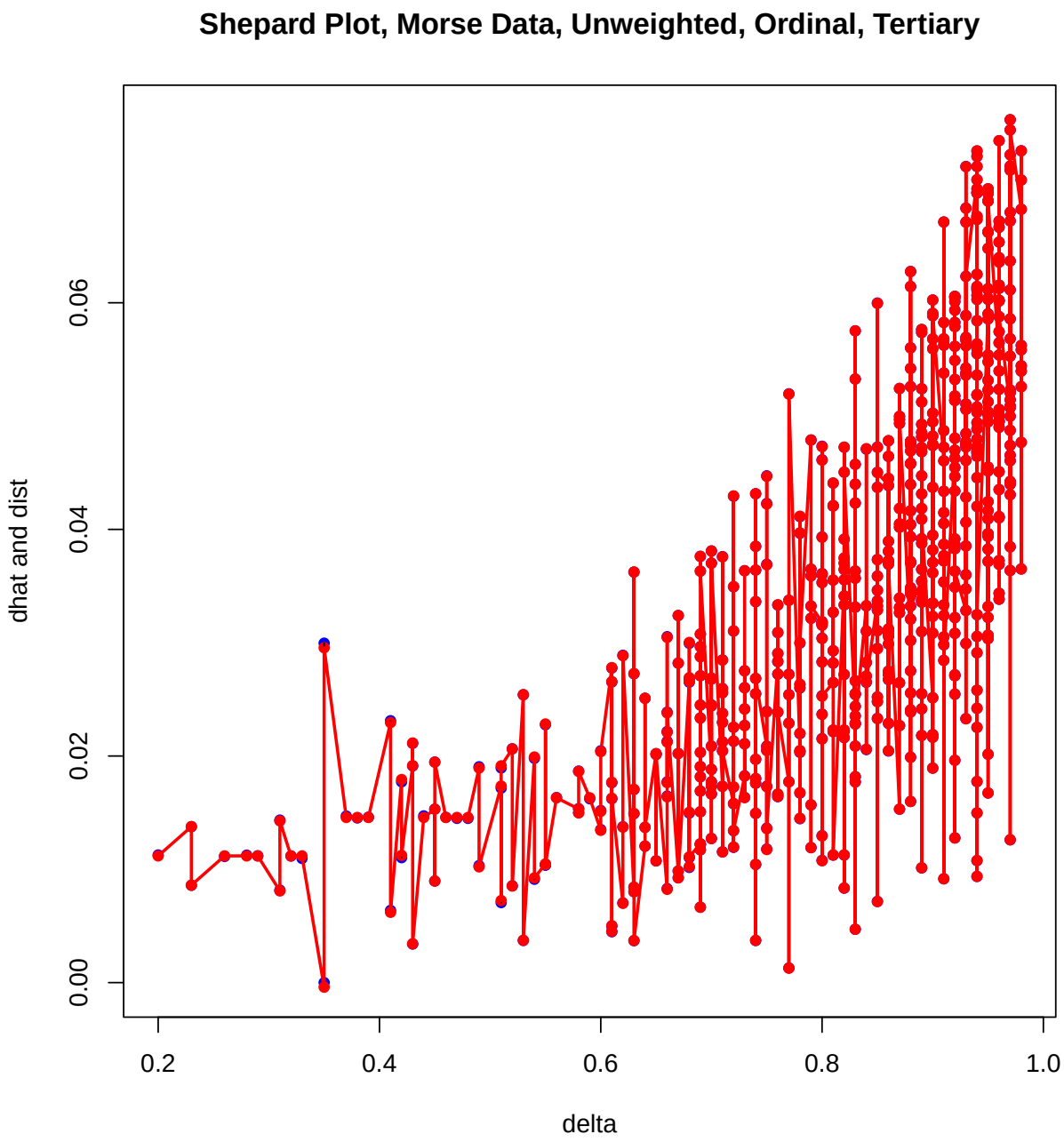

DistDhat Plot, Ekman Data, Unweighted, Ordinal, Secondary
disparity
distance
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14

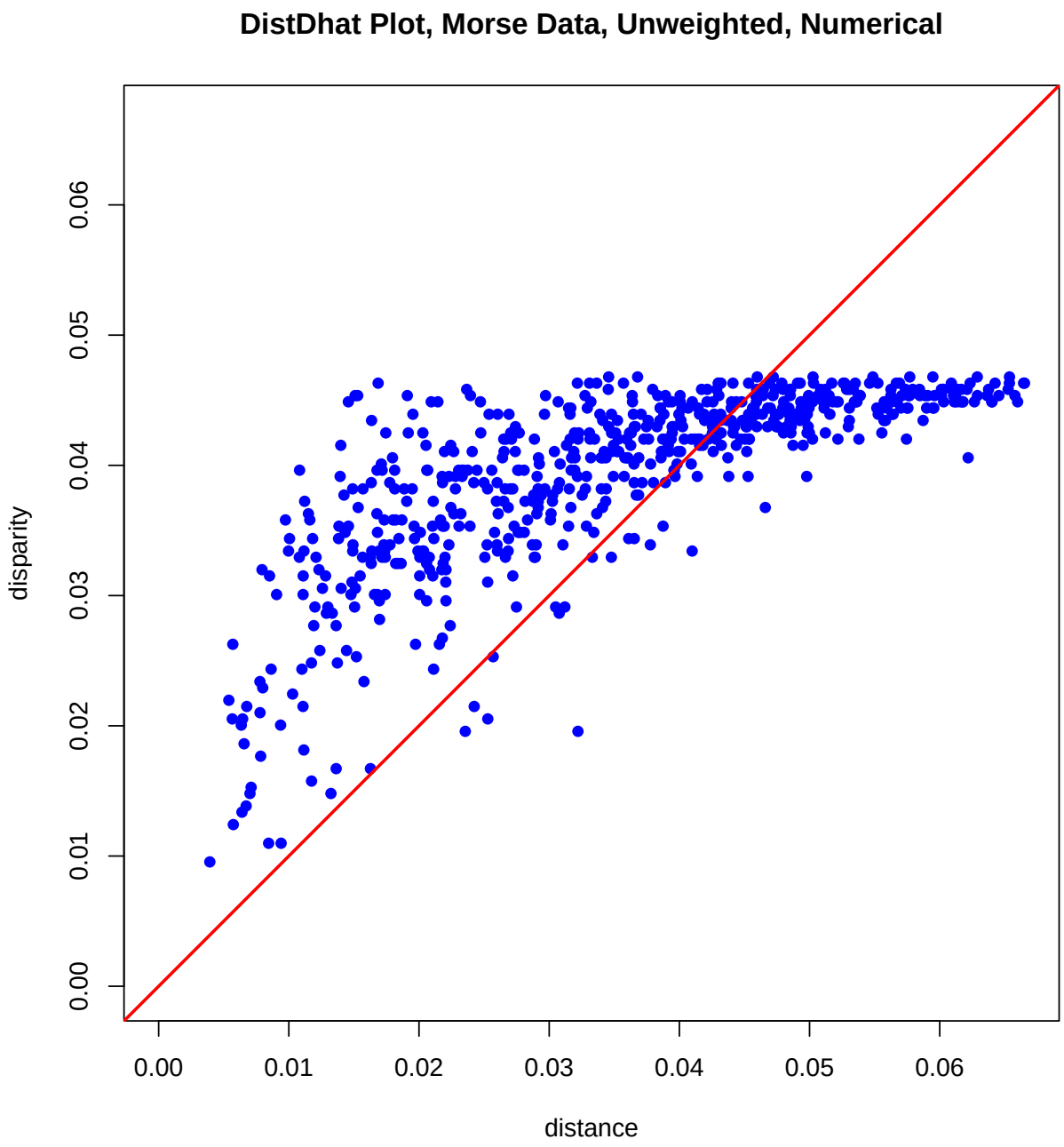

DistDhat Plot, Ekman Data, Unweighted, Ordinal, Tertiary
disparity
0.00
0.05
0.10
0.15
distance
0.00
0.05
0.10
0.15

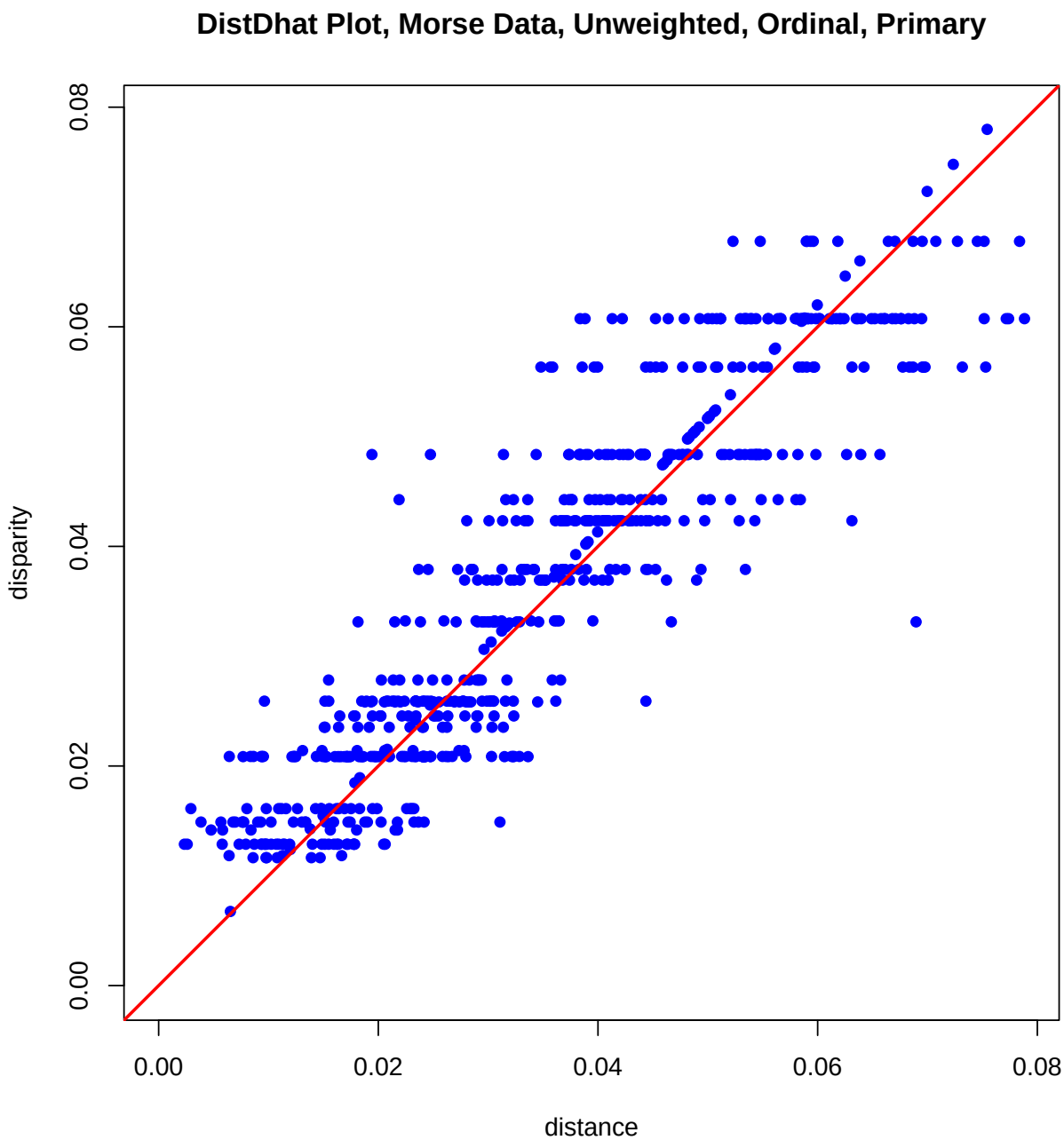

Shepard Plot, Morse Data, Unweighted, Numerical
dhat and dist
0.00
0.01
0.02
0.03
0.04
0.05
0.06
0.2
0.4
0.6
0.8
1.0
delta

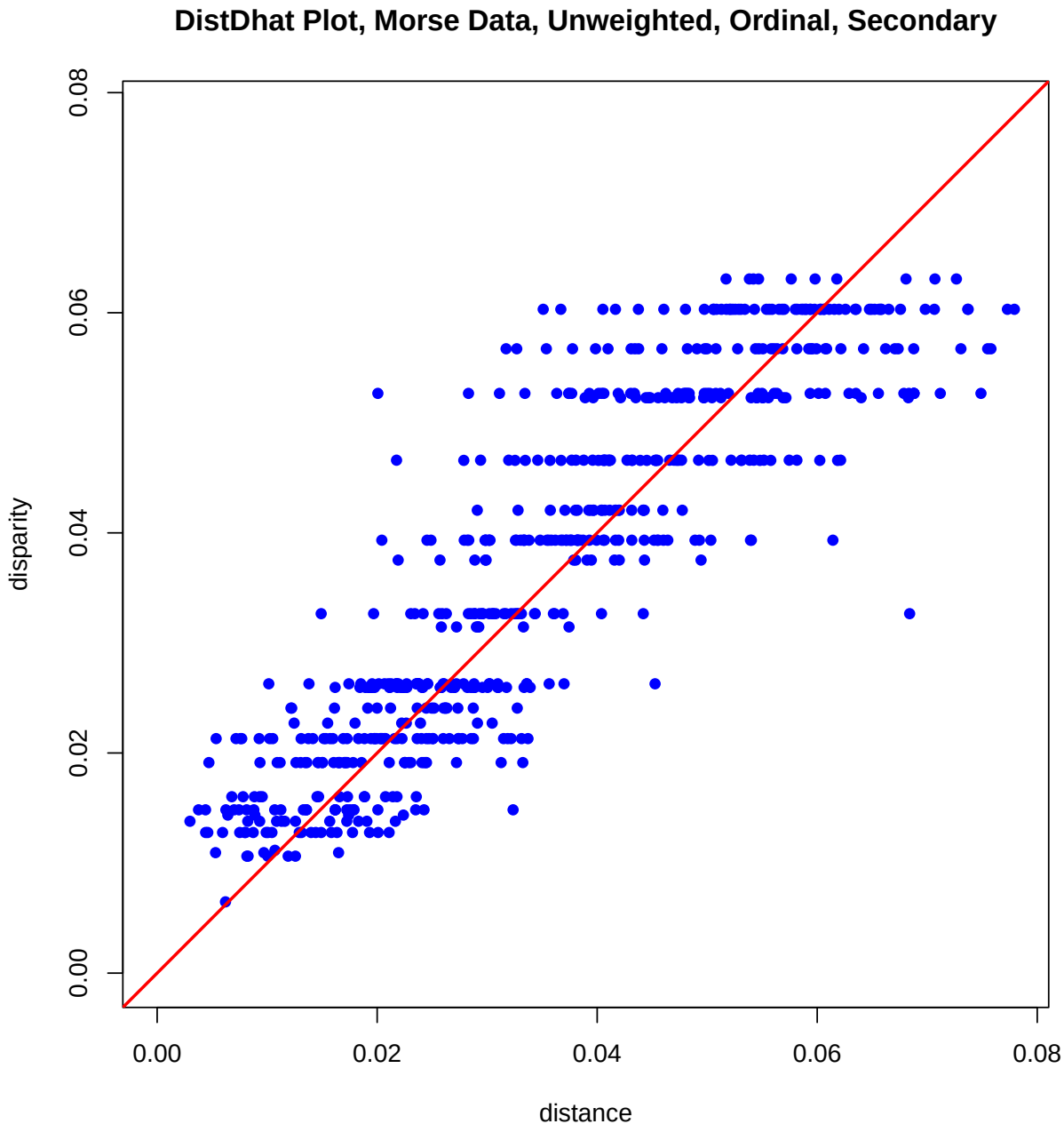

Shepard Plot, Morse Data, Unweighted, Ordinal, Primary
dhat and dist
0.00
0.02
0.04
0.06
0.08
0.2
0.4
0.6
0.8
1.0
delta

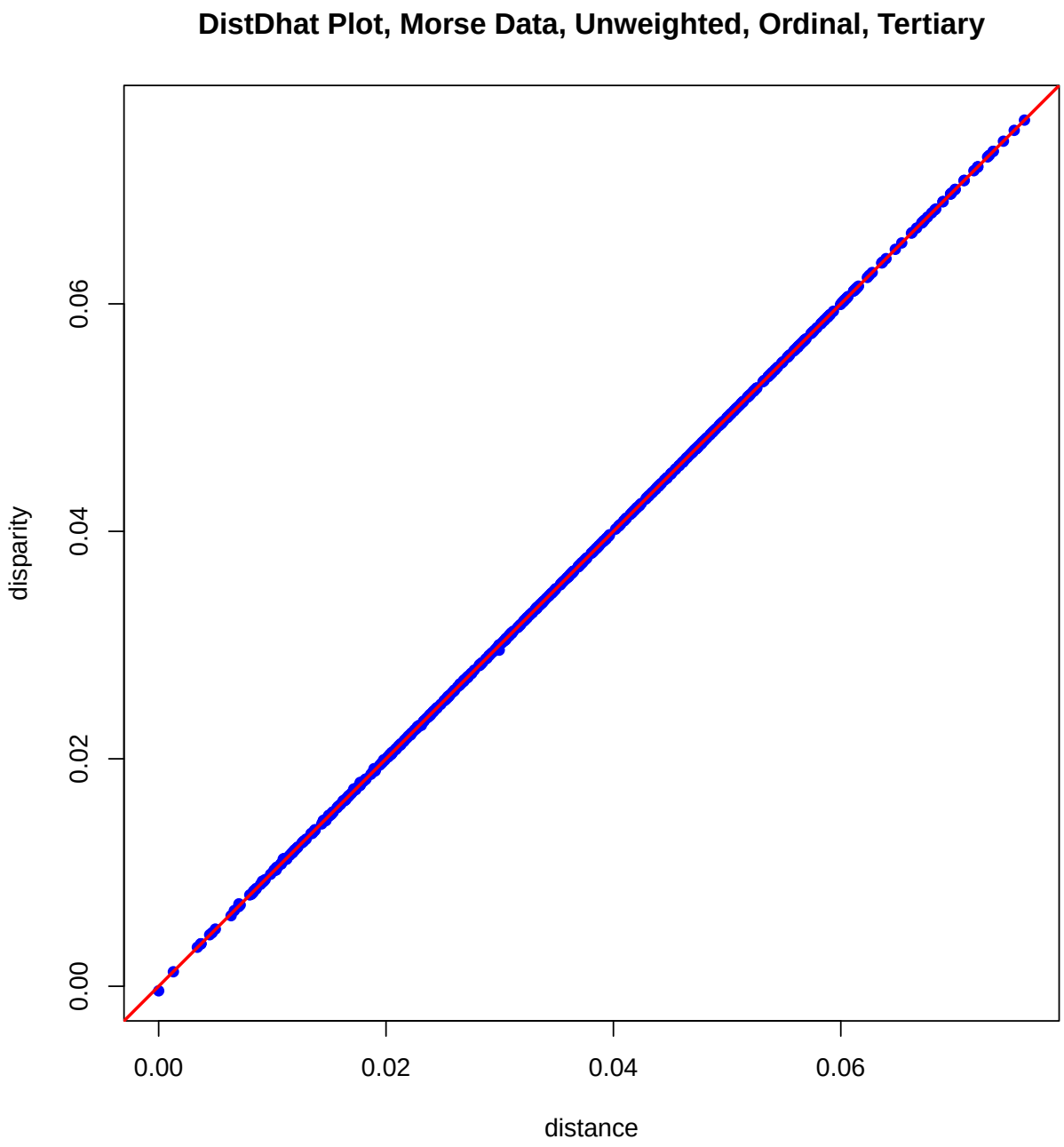

Shepard Plot, Morse Data, Unweighted, Ordinal, Secondary
dhat and dist
0.00
0.02
0.04
0.06
0.08
0.2
0.4
0.6
0.8
1.0
delta


Shepard Plot, Morse Data, Unweighted, Ordinal, Tertiary
dhat and dist
0.00
0.02
0.04
0.06
0.2
0.4
0.6
0.8
1.0
delta


DistDhat Plot, Morse Data, Unweighted, Numerical
disparity
0.00
0.01
0.02
0.03
0.04
0.05
0.06
distance
0.00
0.01
0.02
0.03
0.04
0.05
0.06


DistDhat Plot, Morse Data, Unweighted, Ordinal, Primary
disparity
distance
0.00
0.02
0.04
0.06
0.08


DistDhat Plot, Morse Data, Unweighted, Ordinal, Secondary
disparity
distance
0.00
0.02
0.04
0.06
0.08



References

Anderson, Edgar. 1936. "The Species Problem in Iris." *Annals of the Missouri Botanical Garden* 23: 457–509. https://biostor.org/reference/11559.

Bauschke, Heinz H., Minh N. Bui, and Xianfu Wang. 2018. "Projecting onto the Intersection of a Cone and a Sphere." *SIAM Journal on Optimization* 28: 2158–88.

Borg, Ingwer, and Patrick J. F. Groenen. 2005. *Modern Multidimensional Scaling*. Second Edition. Springer.

Busing, Frank M. T. A. 2022. "Monotone Regression: A Simple and Fast O(n) PAVA Implementation." *Journal of Statistical Software* 102 (Code Snippet 1). https://www.jstatsoft.org/index.php/jss/article/view/v102c01/4306.

De Gruijter, Dato N. M. 1967. *The Cognitive Structure of Dutch Political Parties in 1966*. Report Nos. E019-67. Psychological Institute, University of Leiden.

de Leeuw, Jan. 1968. *Nonmetric Discriminant Analysis*. Research Note Nos. 06-68. Department of Data Theory, University of Leiden. https://deleeuw.github.io/janspubs/deleeuw-r-68-d/deleeuw-r-68-d.pdf.

de Leeuw, Jan. 1975. "A Normalized Cone Regression Approach to Alternating Least Squares Algorithms." Department of Data Theory FSW/RUL. https://deleeuw.github.io/janspubs/deleeuw-u-75-a/deleeuw-u-75-a.pdf.

de Leeuw, Jan. 1977. "Applications of Convex Analysis to Multidimensional Scaling." In *Recent Developments in Statistics*, edited by J. R. Barra, F. Brodeau, G. Romier, and B. Van Cutsem. North Holland Publishing Company. https://deleeuw.github.io/janspubs/deleeuw-c-77/deleeuw-c-77.pdf.

de Leeuw, Jan. 1988. "Convergence of the Majorization Method for Multidimensional Scaling." *Journal of Classification* 5: 163–80. https://deleeuw.github.io/janspubs/deleeuw-a-88-b/deleeuw-a-88-b.pdf.

de Leeuw, Jan. 1994. "Block Relaxation Algorithms in Statistics." In *Information Systems and Data Analysis*, edited by H. H. Bock, W. Lenski, and M. M. Richter. Springer Verlag. https://deleeuw.github.io/janspubs/deleeuw-c-94-c/deleeuw-c-94-c.pdf.

de Leeuw, Jan. 2006. *Accelerated Least Squares Multidimensional Scaling*. Preprint Series No. 493. UCLA Department of Statistics. https://deleeuw.github.io/janspubs/deleeuw-r-06-b/deleeuw-r-06-b.pdf.

de Leeuw, Jan. 2008. *Polynomial Extrapolation to Accelerate Fixed Point Algorithms*. Preprint Series No. 542. UCLA Department of Statistics. https://deleeuw.github.io/janspubs/deleeuw-r-08-i/deleeuw-r-08-i.pdf.

de Leeuw, Jan. 2016. "Gower Rank." https://deleeuw.github.io/janspubs/deleeuw-e-16-k/deleeuw-e-16-k.pdf.

de Leeuw, Jan. 2025a. "Metric/Non-metric Elastic MDS." https://deleeuw.github.io/janspubs/deleeuw-e-25-f/deleeuw-e-25-f.pdf.

de Leeuw, Jan. 2025b. "Metric/Non-metric Sammon Mapping." https://deleeuw.github.io/janspubs/deleeuw-e-25-g/deleeuw-e-25-g.pdf.

de Leeuw, Jan. 2025c. "Squared Distance Scaling." https://deleeuw.github.io/janspubs/deleeuw-e-25-c/deleeuw-e-25-c.pdf.

de Leeuw, Jan, Patrick J. F. Groenen, and Patrick Mair. 2016. "Full-dimensional Scaling." https://deleeuw.github.io/janspubs/deleeuw-groenen-mair-e-16-e/deleeuw-groenen-mair-e-16-e.pdf.

de Leeuw, Jan, and Willem J. Heiser. 1980. "Multidimensional Scaling with Restrictions on the Configuration." In *Multivariate Analysis, Volume V*, edited by P. R. Krishnaiah. North Holland Publishing Company. https://deleeuw.github.io/janspubs/deleeuw-heiser-c-80/deleeuw-heiser-c-80.pdf.

De Leeuw, Jan, Kurt Hornik, and Patrick Mair. 2009. "Isotone Optimization in R: Pool-Adjacent-Violators Algorithm (PAVA) and Active Set Methods." *Journal of Statistical Software* 32 (5): 1–24. https://deleeuw.github.io/janspubs/deleeuw-hornik-mair-a-09/deleeuw-hornik-mair-a-09.pdf.

De Leeuw, Jan, and Patrick Mair. 2009. "Multidimensional Scaling Using Majorization: SMACOF in R." *Journal of Statistical Software* 31 (3): 1–30. https://deleeuw.github.io/janspubs/deleeuw-mair-a-09-c/deleeuw-mair-a-09-c.pdf.

Ekman, Gosta. 1954. "Dimensions of Color Vision." *Journal of Psychology* 38: 467–74.

Fisher, Ronald A. 1936. "The Use of Multiple Measurements in Taxonomic Problems." *Annals of Eugenics* 7: 179–88. https://onlinelibrary.wiley.com/doi/epdf/10.1111/j.1469-1809.1936.tb02137.x.

Groenen, Patrick J. F., W. Glunt, and T. L. Hayden. 1996. *Fast Algorithms for Multidimensional Scaling: A Comparison of Majorization and Spectral Gradient Methods*. Department of Data Theory, University of Leiden.

Groenen, Patrick J. F., and Michel Van de Velden. 2016. "Multidimensional Scaling by

Majorization: A Review." *Journal of Statistical Software* 73 (8): 1–26. https://www.jstatsoft.org/index.php/jss/article/view/v073i08.

Guttman, Louis. 1968. "A General Nonmetric Technique for Fitting the Smallest Coordinate Space for a Configuration of Points." *Psychometrika* 33: 469–506.

Kruskal, Joseph B. 1964a. "Multidimensional Scaling by Optimizing Goodness of Fit to a Nonmetric Hypothesis." *Psychometrika* 29: 1–27.

Kruskal, Joseph B. 1964b. "Nonmetric Multidimensional Scaling: a Numerical Method." *Psychometrika* 29: 115–29.

Kruskal, Joseph B., and J. Douglas Carroll. 1969. "Geometrical Models and Badness of Fit Functions." In *Multivariate Analysis, Volume II*, edited by P. R. Krishnaiah. North Holland Publishing Company.

Lepage-Saucier, Nicolas. 2024. "Alternating Cyclic Vector Extrapolation Technique for Accelerating Nonlinear Optimization Algorithms and Fixed-Point Mapping Applications." *Journal of Computational and Applied Mathematics* 439: 1–27. https://doi.org/10.1016/j.cam.2023.115607.

Mair, Patrick, Patrick J. F. Groenen, and Jan de Leeuw. 2022. "More on Multidimensional Scaling in R: smacof Version 2." *Journal of Statistical Software* 102 (10): 1–47. https://deleeuw.github.io/janspubs/mair-groenen-deleeuw-a-22/mair-groenen-deleeuw-a-22.pdf.

Mersmann, Olaf. 2024. *microbenchmark: Accurate Timing Functions*. https://CRAN.R-project.org/package=microbenchmark.

Qiu, Yixuan, and Jiali Mei. 2024. *RSpectra: Solvers for Large-Scale Eigenvalue and SVD Problems*. https://CRAN.R-project.org/package=RSpectra.

R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. https://www.R-project.org/.

Ramsay, James O. 1975. "Solving Implicit Equations in Psychometric Data Analysis." *Psychometrika* 40: 337–60.

Rosman, Guy, Michael M. Bronstein, Alexander M. Bronstein, Avram Sidi, and Ron Kimmel. 2008. *Fast Multidimensional Scaling Using Vector Extrapolation*. Technical Report CIS-2008 01. Technion - Computer Science Department. https://www.researchgate.net/publication/265141771_Fast_Multidimensional_Scaling_using_Vector_Extrapolation.

Rothkopf, Ernst Z. 1957. "A Measure of Stimulus Similarity and Errors in some Paired-associate Learning." *Journal of Experimental Psychology* 53: 94–101.

Takane, Yoshio. 1977. "On the Relations among Four Methods of Multidimensional Scaling." *Behaviormetrika* 4: 29–42.

Takane, Yoshio, Forrest W. Young, and Jan de Leeuw. 1977. "Nonmetric Individual Differences in Multidimensional Scaling: An Alternating Least Squares Method with Optimal Scaling Features." *Psychometrika* 42: 7–67. https://deleeuw.github.io/janspubs/takane-young-deleeuw-a-77/takane-young-deleeuw-a-77.pdf.

Torgerson, Warren S. 1958. *Theory and Methods of Scaling*. Wiley.

Wish, Myron. 1971. "Individual Differences in Perceptions and Preferences Among Nations." In *Attitude Research Reaches New Heights*, edited by C. W. King and D. Tigert. American Marketing Association.